

**QUIZLY: APPLICATION FOR INTELLIGENT QUIZ AND
ASSESSMENT WITH AI-BASED ANTI-CHEATING
FEATURES**



BACHELORS OF SCIENCE (SOFTWARE ENGINEERING)

SESSION: 2022 - 2026

SUBMITTED BY:

EMAN ANJUM (2225165097)

AYESHA TARIQ (2225165094)

SUPERVISOR:

DR. MUHAMMAD MOHSIN NAZIR

**DEPARTMENT OF SOFTWARE ENGINEERING
LAHORE COLLEGE FOR WOMEN UNIVERSITY**

Lahore, Pakistan

SUBMISSION DATE:

16TH JUNE 2026

Approval Certificate

This is to certify that the Final Year Project titled "**Quizly: An AI-Based Intelligent Quiz and Assessment System with Anti-Cheating Features**" has been completed under our supervision by Eman Anjum (2225165097) and Ayesha Tariq (2225165094), students of the Department of Software Engineering, Lahore College for Women University, Lahore. The project has been reviewed and is approved for submission in partial fulfillment of the requirements for the degree of Bachelor of Science in Software Engineering.

Project Supervisor: Dr. Mohsin Nazir Signature: _____ Date: _____

Head of Department: Signature: _____ Date: _____

Declaration

We, Eman Anjum and Ayesha Tariq students of the Department of Software Engineering, Lahore College for Women University, Lahore, solemnly declare that the Final Year Project titled "**Quizly: An AI-Based Intelligent Quiz and Assessment System with Anti-Cheating Features**" submitted herewith is entirely our own original work. This project has not been submitted, in whole or in part, to any other institution or university for the award of any academic degree, diploma, or qualification.

All sources, references, and literature consulted during the course of this project have been duly acknowledged in accordance with the IEEE citation format. Any similarities with existing work are purely coincidental and unintentional.

Eman Anjum Roll No. 2225165097 Signature: _____ Date: _____

Ayesha Tariq Roll No. 2225165094 Signature: _____ Date: _____

Acknowledgements

All gratitude belongs to Allah Almighty, the Most Gracious and the Most Merciful, whose blessings guided us at every step of this journey and granted us the strength, patience, and clarity of thought to complete this work.

We would like to express our deepest gratitude to our parents and families for their unconditional love, endless encouragement, patience, and unwavering support throughout our academic journey. Their sacrifices, prayers, and belief in our abilities have been a constant source of motivation and strength. This achievement would not have been possible without their guidance and support.

We are deeply and sincerely grateful to our project supervisor, Dr. Mohsin Nazir, whose expert guidance, consistent encouragement, and constructive feedback shaped this project from its earliest conceptual stages through to its completion. His dedication to our academic growth went far beyond what was required, and we are truly fortunate to have had such a mentor.

We would also like to express our heartfelt gratitude to the faculty members of the Department of Software Engineering at Lahore College for Women University. The knowledge and skills we gained throughout our academic program provided the foundation upon which this project was built. Their passion for teaching and commitment to preparing students for real-world challenges has had a lasting impact on us.

Finally, we extend our appreciation to everyone who directly or indirectly contributed to the successful completion of this project. Their support and encouragement are sincerely valued.

Abstract

The growing adoption of e-learning and remote education has made online assessment a critical component of academic institutions worldwide. However, existing digital assessment platforms continue to struggle with fundamental limitations that compromise their effectiveness and reliability. Quiz creation in most platforms remains entirely manual, placing a significant time burden on educators. The evaluation of open-ended and subjective answers lacks automation, resulting in delays and potential inconsistency in grading. Perhaps most critically, anti-cheating controls in widely used platforms are either insufficiently strong or entirely absent, allowing academic dishonesty to undermine the integrity of online examinations.

This project presents **Quizly**, a comprehensive AI-powered quiz and assessment system specifically designed to address these limitations within a single, unified platform. The system integrates Natural Language Processing to automatically generate quiz questions from uploaded lecture material, freeing educators from the repetitive task of manual question writing. An AI-based grading engine evaluates short-answer and descriptive responses by measuring semantic similarity and contextual alignment with model answers, ensuring consistent and unbiased scoring. To protect academic integrity, Quizly incorporates a dedicated anti-cheating module that actively monitors quiz sessions for tab-switching, screenshot attempts, clipboard access, and other suspicious activities, logging all events in real time.

The system is built around three user roles — administrator, teacher, and student — each with a dedicated interface and appropriate access permissions. Development follows the Agile Software Development Life Cycle, enabling iterative delivery and continuous refinement. The technology stack consists of React.js for the frontend, Node.js with Express.js for the backend, and a dual-database approach using MySQL for managing all structured application data, including users, quizzes, questions, quiz attempts, results, generated questions, and activity logs.

Testing is conducted across unit, integration, system, and user acceptance levels to validate functionality, performance, and security. Quizly aims to make online assessment more intelligent, efficient, and trustworthy, supporting educators in delivering fair evaluations and helping institutions uphold academic standards in the digital age.

Keywords: Artificial Intelligence, Natural Language Processing, Online Assessment, Automated Quiz Generation, AI Grading, Anti-Cheating, Academic Integrity, E-Learning.

Table Of Contents

Chapter 1: Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Project Objectives	2
1.4 Scope of the Project.....	3
1.5 Significance of the Project	3
1.5.1 Academic Significance	3
1.5.2 Industrial Significance	4
1.5.3 Social Significance	4
1.6 Report Organization	4
Chapter 2: Literature Review	6
2.1 Introduction	6
2.2 Review of Existing Systems.....	6
2.2.1 Google Forms	6
2.2.2 Kahoot	6
2.2.3 Moodle LMS.....	7
2.2.4 Blackboard Learn	7
2.2.5 Canvas LMS	7
2.2.6 AI-Based Assessment Research Systems	8
2.2.7 Online Proctoring Systems	8
2.3 Comparative Analysis	8
2.4 Research Gap Identification	9
2.5 Summary	10
Chapter 3: System Analysis and Design	11
3.1 Requirement Analysis	11
3.1.1 Functional Requirements	11
3.1.2 Non-Functional Requirements.....	14
3.2 Feasibility Study.....	17
3.2.1 Technical Feasibility.....	17
3.2.2 Economic Feasibility	18
3.2.3 Operational Feasibility	18

3.2.4 Legal Feasibility	18
3.3 Use Case Diagram	18
3.4 Activity Diagrams	20
3.5 Sequence Diagrams	22
3.6 Class Diagram	24
3.7 Database Design	26
3.8 System Architecture	27
3.9 User Interface Design	27
3.10 SDLC Model — Agile Methodology	34
3.11 Project Timeline	34
3.12 Risk Analysis	36
Chapter 4: Implementation	40
4.1 Development Environment	40
4.2 Technologies Used	40
4.3 System Modules	42
4.3.1 Authentication Module	42
4.3.2 Quiz Management Module	42
4.3.3 NLP Quiz Generation Module	42
4.3.4 AI Grading Module	42
4.3.5 Anti-Cheating Module	43
4.3.6 Reporting Module	43
4.4 Database Implementation	43
4.5 Key Algorithms and Methods	44
4.5.1 NLP Quiz Generation Algorithm	44
4.5.2 AI Grading Algorithm	46
4.5.3 Anti-Cheating Monitoring Algorithm	48
4.6 Screenshots of Implementation	50
Chapter 5: Testing and Results	55
5.1 Testing Strategy	55
5.2 Unit Testing	55
5.3 Integration Testing	56
5.4 System Testing	58
5.5 User Acceptance Testing	60
5.6 Results and Discussion	62
Chapter 6: Conclusion and Future Work	63

6.1 Conclusion.....	63
6.2 Achievements	64
6.3 Limitations	65
6.4 Future Enhancements	65
References	66
Appendices	69
Appendix A: User Manual	69
Appendix B: Source Code Samples	70
Appendix C: Survey Questionnaire.....	71
Appendix D: Test Cases	72
Appendix E: Deployment Evidence And Hosting Details	73

List of Figures:

Figure 3.1: Use Case Diagram	19
Figure 3.2: Teacher Activity Diagram	20
Figure 3.3: Student Activity Diagram.....	21
Figure 3.4: Administrator Diagram.....	22
Figure 3.5: User Login Diagram.....	23
Figure 3.6: Quiz Attempt Diagram	24
Figure 3.7: Class Diagram	25
Figure 3.8: Login Page.....	28
Figure 3.9: Teacher Dashboard.....	28
Figure 3.10: Quiz Creation	29
Figure 3.11: NLP Interface	30
Figure 3.12: Student Dashboard	31
Figure 3.13: Quiz Attempt Interface.....	32
Figure 3.14: Quiz Result	33
Figure 3.15: Admin Panel	33
Figure 3.16: Gantt Chart	36
Figure 4.1: Flow Chart 1	45
Figure 4.2: Flow Chart 2	47
Figure 4.3: Flow Chart 3	49
Figure 4.4: LogIn Page	50
Figure 4.5: Teacher Dashboard.....	51
Figure 4.6: Manual Quiz Creation	51
Figure 4.7: QuizQuestion Management.....	52
Figure 4.8: Anti Cheating Feature	52
Figure 4.9: Anti Cheating Notification	53
Figure 4.10: Student Result Page.....	53
Figure 4.11: New Teacher Registration	54
Figure 4.12: Admin Panel	54
Figure E.1: Repository Creation	74
Figure E.2: Uploaded Source Code	74
Figure E.3: Database Deployment	75
Figure E.4: Backend Deployment.....	76
Figure E.5: Frontend Deployment	76

List of Tables:

Table 2.1: Comparative Analysis.....	9
Table 3.1: Functional Requirements	11
Table 3.2: Non-functional Requirements.....	15
Table 3.3: Feasibility Study	17
Table 3.4: Entites,Keys & Realtionships	26
Table 3.5: Project Phases	35
Table 3.6: Risk Analysis Matrix	37
Table 4.1: Used Technologies.....	41
Table 5.1: Unit Test Cases	56
Table 5.2: Integration Test Cases.....	57
Table 5.3: System Test Cases	60
Table 5.4: UAT Teacher Feedback.....	61
Table 5.5: UAT Student Feedback.....	62
Table D.1: Sample Test Cases	73

Chapter 1: Introduction

1.1 Background

The last decade has brought about a fundamental transformation in how education is delivered and assessed. The widespread proliferation of internet access, coupled with the development of sophisticated digital learning platforms, has shifted a significant portion of academic activity from physical classrooms to online environments. This transition has been further accelerated by global disruptions such as the COVID-19 pandemic, which forced institutions across the world to adopt online alternatives almost overnight. Even as physical education has resumed in many regions, the digital tools and practices developed during this period have remained deeply embedded in academic culture.

Online assessment, in particular, has become a cornerstone of how educational institutions measure student learning. Digital quizzes, assignments, and examinations offer a range of practical advantages over their paper-based counterparts. They are accessible from any location, can be administered to large numbers of students simultaneously, and produce immediate digital records that are easy to store and analyze. For institutions managing hundreds or thousands of students, the logistical convenience of online assessment is undeniable.

However, the adoption of online assessment has also exposed a set of persistent and serious challenges that existing platforms have been slow to resolve. Most commercially available quiz platforms are built around multiple-choice and true/false formats, which are easy to grade automatically but limited in their ability to evaluate higher-order thinking skills such as analysis, synthesis, and critical reasoning. When assessments require written responses which are essential for measuring true comprehension grading still depends overwhelmingly on manual human effort. This is time-consuming, difficult to scale, and introduces the risk of evaluator bias and inconsistency.

Security is an equally serious concern. Without robust anti-cheating mechanisms, online examinations are vulnerable to a wide range of dishonest behaviors. Students can switch browser tabs to access external resources, copy text from external sources using the clipboard, take screenshots of questions to share with others, or simply navigate away from the exam entirely. Most current platforms detect none of these behaviors, creating a system where academic dishonesty can occur with little risk of detection.

Recent developments in Artificial Intelligence and Natural Language Processing have opened new possibilities for addressing these longstanding problems. Modern AI systems can understand the meaning of text, evaluate the relevance of an answer relative to a model, and generate content automatically from source material. Applying these capabilities to the domain of educational assessment offers a path toward systems that are simultaneously smarter, fairer, and more secure. This is the opportunity that Quizly is designed to realize.

1.2 Problem Statement

Despite the widespread availability of online quiz and assessment tools, a careful examination of the current landscape reveals that no existing platform adequately addresses the full range of challenges involved in conducting fair, efficient, and secure digital examinations. The specific problems that motivate this project are as follows:

- **Manual Quiz Creation:** In virtually all widely used platforms, the creation of quiz content is a manual process. Teachers must write each question individually, define answer options, assign marks, and configure question-level settings one by one. For large courses with frequent assessments, this represents an enormous investment of time that could otherwise be directed toward instruction and student support.
- **Absence of Automated Subjective Grading:** Objective questions such as multiple choice and true/false can be graded automatically with high accuracy. However, when assessments include short-answer or descriptive questions which are necessary for testing deeper understanding there is no automated mechanism available in mainstream platforms. These questions must be reviewed and graded manually by the teacher, which introduces delays of days or even weeks between submission and feedback.
- **Weak Anti-Cheating Controls:** The security measures in most online assessment platforms are insufficient for formal academic examinations. Students can easily switch between browser tabs or applications during a quiz, access external resources through the same device, copy question text to share externally, or navigate away from the exam without automatic detection. This creates a fundamentally unfair testing environment in which dishonest students gain an advantage over honest ones.
- **Delayed and Inconsistent Feedback:** The combination of manual grading and limited automation means that students frequently wait extended periods to receive their results. Additionally, because subjective questions are graded manually, there is a risk that the same answer receives different marks depending on the evaluator or even the time of day the evaluation is performed.
- **Scalability Limitations:** As student numbers increase, the problems described above become more severe. Manual quiz creation and grading simply do not scale, and without automation, the quality of assessment inevitably suffers as workload increases.

Taken together, these problems represent a clear and significant gap in the current state of online assessment technology. There is a genuine need for a platform that can automate the creation and evaluation of quizzes, enforce examination security, and do all of this within a system that is practical and accessible for everyday educational use.

1.3 Project Objectives

The following objectives guide the design and development of Quizly:

Primary Objective: To design, develop, and validate an AI-powered online quiz and assessment system that integrates automated question generation, intelligent subjective grading, and robust anti-cheating mechanisms within a single, role-based web application.

Specific Objectives:

- To conduct a comprehensive review of existing assessment platforms and AI-based grading research in order to identify functional limitations and research gaps.
- To develop an NLP-based question generation module that automatically produces quiz questions from uploaded lecture notes and course material, reducing manual preparation time for teachers.
- To implement an AI-based grading engine that evaluates short-answer and descriptive student responses using semantic similarity analysis, ensuring consistent and unbiased scoring.

- To design and integrate a real-time anti-cheating module that detects and logs suspicious behaviors including tab-switching, screenshot attempts, and clipboard access during quiz sessions.
- To build a secure, scalable, and role-based web application supporting administrators, teachers, and students with appropriate access controls and dedicated interfaces.
- To generate detailed performance reports and analytics that give teachers meaningful insight into student learning and help identify areas requiring instructional attention.
- To validate the system through comprehensive testing including unit, integration, system, and user acceptance testing.

1.4 Scope of the Project

Included Features

The following functionalities are within the defined scope of this project:

- User registration, login, and role-based access control for three user types: Administrator, Teacher, and Student.
- Manual quiz creation supporting multiple-choice, true/false, and short-answer question types with configurable time limits and scheduling.
- Automated quiz generation from uploaded lecture material using Natural Language Processing.
- AI-driven evaluation and scoring of short-answer student responses.
- A real-time anti-cheating module covering tab-switch detection, screenshot prevention, clipboard restriction, and activity logging.
- Immediate result generation and display upon quiz submission.
- Performance reporting and analytics dashboards for teachers.
- Administrative user management and system monitoring functions.

Excluded Features

The following are outside the current project scope and are identified as future work:

- A dedicated mobile application for Android or iOS.
- Biometric proctoring features including facial recognition and eye-tracking.
- Voice-based assessment or speech recognition grading.
- Offline examination support without an active internet connection.
- Direct integration with third-party Learning Management Systems.

1.5 Significance of the Project

1.5.1 Academic Significance

This project provides direct and meaningful application of concepts studied across the Software Engineering curriculum. The system draws on knowledge from machine learning, software architecture design, relational and non-relational database systems, web security, human-computer interaction, and software testing. By applying these disciplines to a real and unsolved problem in the education sector, the project demonstrates practical software engineering competence and contributes to research in the emerging field of AI-assisted educational technology. The challenges addressed automated question generation, semantic grading, and

real-time security monitoring are active areas of academic inquiry, and the solutions developed in this project add to the body of practical knowledge in these areas.

1.5.2 Industrial Significance

From an industry perspective, Quizly addresses a genuine market need. Educational institutions, corporate training departments, and online learning platforms all require assessment tools that are more automated, consistent, and secure than current offerings. The system is built entirely on open-source technologies, making it financially accessible and technically adaptable for institutions of varying sizes and resources. By automating labor-intensive tasks such as question creation and subjective grading, Quizly reduces operational costs and allows teaching staff to focus their efforts on instruction rather than administration. The modular architecture also ensures that the platform can be extended or scaled as institutional needs evolve.

1.5.3 Social Significance

At a broader societal level, the integrity of educational assessment has direct implications for fairness, merit, and public trust in academic credentials. When online examinations lack adequate security, students who cheat gain an unfair advantage over those who do not, which undermines the value of academic qualifications for everyone. Quizly addresses this directly by making cheating significantly more difficult to execute undetected. Furthermore, by providing immediate and consistent feedback to students, the system supports better learning outcomes and reduces the anxiety associated with waiting for results. In regions where access to high-quality educational infrastructure is limited, a scalable and intelligent assessment platform like Quizly can meaningfully contribute to improving educational equity and quality.

1.6 Report Organization

This report is organized into six chapters, each addressing a distinct phase of the project lifecycle.

Chapter One — Introduction provides the background context for the project, defines the problem being solved, establishes the objectives, outlines the scope, and explains the significance of the work from academic, industrial, and social perspectives.

Chapter Two — Literature Review presents a detailed examination of existing online assessment platforms and relevant academic research. It includes a comparative analysis of reviewed systems and clearly identifies the research gaps that this project addresses.

Chapter Three — System Analysis and Design covers the complete technical design of the system. It includes requirement analysis, feasibility study, UML diagrams, database design, system architecture, and user interface design.

Chapter Four — Implementation describes how the designed system was built, covering the development environment, technologies used, module implementation details, database structure, core algorithms, and screenshots of the working application.

Chapter Five — Testing and Results presents the testing strategy and outcomes across unit, integration, system, and user acceptance levels, followed by a discussion of results.

Chapter Six — Conclusion and Future Work summarizes the achievements of the project, acknowledges its current limitations, and proposes directions for future enhancement and research.

Chapter 2: Literature Review

2.1 Introduction

The field of educational technology has evolved rapidly over the past two decades, producing a wide range of tools and platforms intended to support digital learning and assessment. From basic survey tools adapted for quiz creation to sophisticated Learning Management Systems and AI-powered grading engines, the landscape is diverse. However, diversity alone does not imply completeness as this review will demonstrate, each category of existing solution addresses certain aspects of the assessment problem while leaving others largely unresolved.

This chapter presents a structured review of the most widely used online assessment platforms, followed by a discussion of relevant research in AI-based grading and online proctoring. The chapter concludes with a comparative analysis and the identification of the specific research gaps that motivate the development of Quizly.

The review covers tools including Google Forms, Kahoot, Moodle, Blackboard, Canvas LMS, AI-based assessment research systems, and dedicated online proctoring platforms. For each system, the key features, strengths, and limitations are examined in relation to the four core requirements of an ideal assessment platform: automated quiz generation, AI-based subjective grading, anti-cheating enforcement, and comprehensive reporting.

2.2 Review of Existing Systems

2.2.1 Google Forms

Google Forms is a free online survey and quiz creation tool developed by Google as part of the Google Workspace suite. It is widely used in academic settings due to its accessibility and ease of use. Teachers can create quizzes consisting of multiple question types including multiple choice, short answer, paragraph, and dropdown. The platform supports automatic grading for objective question types and allows the assignment of point values to questions.

Despite its popularity, Google Forms falls significantly short as a formal academic assessment tool. It provides no mechanism for detecting or preventing cheating students can freely switch browser tabs, search for answers online, or share the quiz link with others. There is no AI-based functionality for question generation or subjective answer evaluation. All open-ended responses must be reviewed and graded manually. The analytics available are basic, limited to response summaries without deeper performance insights. For casual classroom use, Google Forms is adequate, but for any assessment where security and fairness are priorities, it is fundamentally inadequate.

2.2.2 Kahoot

Kahoot is a game-based learning platform designed to make classroom engagement interactive and enjoyable. It allows teachers to create multiple-choice quizzes that students participate in competitively in real time, answering through a web browser or mobile device. The gamified format featuring live leaderboards, timed responses, and colorful visuals has proven effective at motivating participation, particularly in younger student groups.

However, Kahoot's design makes it unsuitable for formal academic evaluation. It supports only multiple-choice questions and awards points based on both correctness and response speed, which is inappropriate for serious assessment contexts. There is no support for subjective answer types, no AI functionality, no meaningful anti-cheating controls, and no detailed performance analytics. Kahoot functions effectively as an engagement and review tool, but it cannot substitute for a genuine assessment system.

2.2.3 Moodle LMS

Moodle is an open-source Learning Management System used by educational institutions worldwide for course management, content delivery, and assessment. It is one of the most feature-rich platforms reviewed, supporting a wide range of question types, configurable quiz settings, role-based access control, and integration with a large library of third-party plugins. Teachers can create question banks, schedule assessments, set time limits, and access detailed student activity logs.

Despite its breadth of features, Moodle has significant practical limitations. The platform requires substantial technical expertise to install, configure, and maintain, making it challenging for institutions without dedicated IT staff. While Moodle supports AI-based grading and anti-cheating through plugins, these are not native capabilities they require additional installation, configuration, and maintenance. The native quiz engine supports automatic grading only for objective questions. Subjective answer evaluation remains manual unless appropriate plugins are correctly configured. Furthermore, Moodle's interface, while comprehensive, can be unintuitive for new users, resulting in a steep learning curve.

2.2.4 Blackboard Learn

Blackboard Learn is a commercial Learning Management System widely used at the university level, particularly in North America and Europe. It offers comprehensive course management features including grade books, discussion boards, assessment tools, and content organization. Blackboard supports various question formats and provides some analytics for tracking student performance.

As a proprietary commercial platform, Blackboard requires significant licensing fees, which places it out of reach for many smaller institutions. Like Moodle, it does not natively support AI-based question generation or subjective grading. Anti-cheating features are limited without the integration of a separate proctoring solution. The platform's complexity and cost create accessibility barriers, and its native assessment capabilities do not substantially exceed what open-source alternatives like Moodle offer.

2.2.5 Canvas LMS

Canvas by Instructure is another widely adopted Learning Management System that offers a cleaner and more modern interface compared to Blackboard or Moodle. It is used by universities and K-12 institutions and supports quiz creation, assignment management, grade tracking, and integration with third-party tools through a well-documented API.

Canvas shares many of the same limitations as other LMS platforms with respect to AI grading and assessment security. It does not natively generate quiz questions from course material, and subjective answer evaluation requires manual grading. While Canvas supports integration with

some proctoring services, these are separate commercial products that add cost and complexity. The platform's assessment capabilities are solid for standard use cases but do not address the intelligent automation requirements that define this project.

2.2.6 AI-Based Assessment Research Systems

Academic research on AI-based assessment has grown considerably over the past decade, driven by advances in NLP and machine learning. Several research systems have demonstrated encouraging results in automatically grading short-answer and essay responses. Systems built on models such as BERT and its variants, along with sentence embedding frameworks and semantic similarity measures, have shown that machine evaluation of written responses can approach human-level accuracy for well-defined question types in specific subject domains.

Research efforts such as the Automated Student Assessment Prize (ASAP) competitions and subsequent published work have established benchmarks for automated essay scoring and short-answer grading. Studies have consistently shown that AI models can evaluate semantic content more reliably than keyword-based matching, and that with adequate training data, these models generalize reasonably well across different student responses to the same question.

However, these research systems share a critical limitation: they are academic prototypes, not production-ready applications. They typically lack user interfaces, role-based access systems, security controls, and the kind of practical integration required for institutional adoption. Many require significant computational resources and carefully curated training datasets. The gap between demonstrating accuracy in a controlled research setting and deploying a fully functional application in a real educational environment remains wide.

2.2.7 Online Proctoring Systems

Online proctoring platforms such as ProctorU, ExamSoft, and Honorlock have been developed specifically to address examination security in remote settings. These systems use webcam recordings, screen capture, keystroke logging, and in some cases AI-driven behavior analysis to monitor students during examinations. Flagged sessions are reviewed by human proctors or automatically analyzed for suspicious patterns.

These platforms represent the strongest available solutions for examination security. However, they are entirely focused on surveillance and monitoring they offer no functionality related to quiz creation, question generation, or answer evaluation. They are designed to work alongside existing assessment platforms rather than replacing them, which means institutions must maintain multiple separate systems. Additionally, proctoring platforms raise concerns about student privacy, can create test anxiety, and typically carry substantial per-use or subscription costs that are prohibitive for many institutions. Their constant monitoring approach has also faced criticism from student advocacy groups.

2.3 Comparative Analysis

The following table presents a structured comparison of all reviewed systems against the key capability dimensions relevant to this project.

Comparative Analysis of Existing Online Assessment Systems

Feature	Google Forms	Kahoot	Moodle	Blackboard	Canvas	AI Research Systems	Proctoring Tools	Quizly (Proposed)
AI-Based Grading	No	No	Limited	Limited	Limited	Yes	No	Yes
NLP Quiz Generation	No	No	No	No	No	Partial	No	Yes
Subjective Evaluation	No	No	Manual	Manual	Manual	Yes	No	Yes
Anti-Cheating Controls	None	None	Moderate	Moderate	Moderate	None	Strong	Strong
Role-Based Access	Basic	No	Yes	Yes	Yes	No	Limited	Yes
Performance Analytics	Basic	Basic	Good	Good	Good	Limited	Limited	Advanced
Ease of Use	High	High	Low	Low	Medium	Low	Medium	High
Open Source / Free	Yes	Freemium	Yes	No	No	N/A	No	Yes

Table 2.1: Comparative Analysis

2.4 Research Gap Identification

The review of existing systems makes several gaps in the current literature and toolset clearly apparent.

Gap 1 — Absence of Integrated NLP-Based Question Generation: None of the reviewed mainstream platforms offer native functionality for generating quiz questions automatically from course material. This remains an entirely manual process across all widely used systems, despite the demonstrated feasibility of NLP-based question generation in academic research settings.

Gap 2 — Lack of Automated Subjective Answer Evaluation in Production Systems: While research has demonstrated the viability of AI-based grading for short and long-form responses, no mainstream assessment platform has successfully integrated this capability into a production-ready, accessible application. The gap between research prototype and deployable product remains unaddressed.

Gap 3 — Fragmented Security and Assessment Functionality: Anti-cheating mechanisms are available through dedicated proctoring platforms, but these are entirely separate from quiz

creation and grading tools. Institutions that need both must maintain and pay for multiple separate systems. There is no single platform that combines assessment automation with integrated security.

Gap 4 — Absence of a Unified, Intelligent Assessment Platform: The fundamental gap identified across all reviewed systems is the absence of a single, unified platform that combines intelligent quiz generation, automated subjective grading, and robust anti-cheating within an accessible, easy-to-use application. This is precisely the gap that Quizly is designed to fill.

2.5 Summary

This chapter has reviewed six categories of existing assessment-related platforms and systems, covering tools from simple quiz creators to sophisticated LMS platforms, AI research systems, and dedicated proctoring services. The review consistently reveals a fragmented landscape in which each available solution addresses one or two dimensions of the assessment challenge while leaving others unaddressed. The comparative analysis and gap identification confirm that there is a clear and significant need for an integrated, intelligent, and secure assessment platform. Quizly is proposed as a direct response to this need, combining the capabilities that existing solutions handle individually into a single, unified system.

Chapter 3: System Analysis and Design

3.1 Requirement Analysis

The requirements for Quizly were gathered through a combination of existing system analysis, review of related research, and examination of the specific problems identified in the problem statement. Requirements are organized into functional and non-functional categories.

3.1.1 Functional Requirements

Functional requirements describe the specific actions, behaviors, and operations the system must perform. Each requirement has been assigned a unique identifier for traceability throughout the design, implementation, and testing phases.

Functional Requirements of the Quizly System

Req. ID	Requirement Title	Description	Priority
FR-01	User Registration	System shall allow new users to register with name, email, password, and role	High
FR-02	User Authentication	System shall authenticate users and issue JWT on successful login	High
FR-03	Role-Based Access Control	System shall restrict access based on user role: Admin, Teacher, Student	High
FR-04	Manual Quiz Creation	Teachers shall create quizzes with MCQ, True/False, and short-answer questions	High
FR-05	Automated Quiz Generation	System shall generate questions from uploaded lecture documents using NLP	High
FR-06	Quiz Scheduling and Assignment	Teachers shall schedule quizzes and assign them to student groups	Medium
FR-07	Secure Quiz Attempt	Students shall attempt quizzes in a monitored, browser-locked environment	High
FR-08	Automated Objective Grading	System shall auto-grade MCQ and True/False questions upon submission	High
FR-09	AI-Based Subjective Grading	System shall evaluate short-answer responses using semantic similarity scoring	High
FR-10	Anti-Cheating Monitoring	System shall detect and log tab-switch, screenshot, and clipboard events	High
FR-11	Result Generation and Display	System shall display instant results after quiz submission	High
FR-12	Teacher Reporting Dashboard	Teachers shall access class-wide analytics and per-student result breakdowns	Medium
FR-13	Administrative User Management	Admins shall manage accounts, roles, and system activity logs	Medium

Table 3.1: Functional Requirements

The following section provides a detailed explanation of each functional requirement.

FR-01: User Registration

The system shall provide a registration interface accessible to new users. During registration, the user must supply their full name, a valid email address, a password, and their intended role within the system. All submitted data shall be validated on both the client side and the server side before being processed. Email addresses must be unique across the system duplicate registrations using the same email shall be rejected with an appropriate error message. Passwords shall meet a defined minimum length and complexity requirement before being accepted. Upon successful validation, the password shall be hashed using bcrypt before being stored in the database. No plaintext passwords shall ever be persisted.

FR-02: User Authentication

The system shall authenticate users by verifying their submitted email address and password against stored credentials. Upon receiving a login request, the backend shall retrieve the corresponding user record from the database and use bcrypt's comparison function to verify the submitted password against the stored hash. If authentication succeeds, the system shall generate a JSON Web Token containing the user's ID and role as payload claims, sign it with a server-side secret key, and return it to the client. This token shall be used to authorize all subsequent requests to protected API endpoints. Failed login attempts shall produce a generic error response that does not reveal whether the failure was due to an unrecognized email or an incorrect password, in order to prevent user enumeration attacks.

FR-03: Role-Based Access Control

The system shall enforce strict access restrictions based on the authenticated user's assigned role. Three roles are defined: Administrator, Teacher, and Student. Every API endpoint that serves protected resources shall be guarded by middleware that validates the incoming JWT and checks the role claim against the permission requirements for that endpoint. Administrators shall have access to user management functions and system-wide activity logs. Teachers shall have access to quiz creation, NLP generation, result viewing, and reporting features. Students shall have access only to their assigned quizzes and their own personal results. Any request from a user attempting to access a resource outside their role's permissions shall be rejected with a 403 Forbidden response.

FR-04: Manual Quiz Creation

Teachers shall be able to create quizzes manually through a dedicated quiz creation interface. The teacher must provide a quiz title, an optional description, a time limit in minutes, and a publication window defining the start and end dates during which the quiz will be accessible to students. The teacher shall then add individual questions using the question editor. Three question types are supported: multiple-choice questions with up to four answer options and one correct answer, true/false questions with two fixed options, and short-answer questions requiring a model answer to be entered by the teacher for AI grading purposes. Each question must be assigned a marks value. The teacher may add, edit, reorder, or delete questions at any point before the quiz is published.

FR-05: Automated Quiz Generation

The system shall provide teachers with the option to generate quiz questions automatically from uploaded course material. The teacher shall upload a document file through the NLP upload interface. The system shall extract the text content from the document and pass it through the NLP question generation pipeline, which processes the content, scores sentences by relevance, and formulates candidate questions. The generated questions shall be presented to the teacher in a review interface where each question can be accepted, edited, or discarded before being added to the quiz. Only questions that have been reviewed and accepted by the teacher shall be included in the final published quiz.

FR-06: Quiz Scheduling and Assignment

Teachers shall be able to configure the availability window of each quiz by setting a specific start date and time and an end date and time. Students shall only be able to access and attempt a quiz during its active window. Outside this window, the quiz shall appear in the student's list but shall be marked as either upcoming or closed, and the attempt interface shall not be accessible. Teachers shall also be able to assign quizzes to specific student groups or individual student accounts. Unassigned students shall not see the quiz in their dashboard regardless of the quiz's active status.

FR-07: Secure Quiz Attempt

Students shall be able to access assigned quizzes through their dashboard and begin an attempt during the active window. Upon beginning a quiz, the system shall load the quiz interface and simultaneously activate the anti-cheating module. The quiz interface shall display questions and collect responses from the student. A countdown timer shall be prominently displayed throughout the attempt. When the configured time limit expires, the system shall automatically submit the student's current responses without requiring manual action. Students who submit before time expires shall do so through a confirmation-prompted submit button. Once submitted, the student shall not be able to return to modify their responses.

FR-08: Automated Objective Grading

Upon quiz submission, the system shall immediately and automatically grade all multiple-choice and true/false questions by comparing the student's selected answer against the stored correct answer for each question. Full marks shall be awarded for a correct selection. Zero marks shall be awarded for an incorrect selection. Partial marks shall not apply to objective question types. The computed scores for all objective questions shall be aggregated and stored in the results record for the attempt.

FR-09: AI-Based Subjective Grading

For short-answer questions, the system shall route the student's response to the AI grading module along with the teacher-provided model answer. The module shall preprocess both texts, generate semantic vector embeddings, compute the cosine similarity between the vectors, and assign a proportional score based on the degree of semantic alignment. The computed score shall be normalized to the marks allocated for that question and stored alongside the response record. The AI-assigned score shall be included in the student's total result and displayed on the results page.

FR-10: Anti-Cheating Monitoring

The anti-cheating module shall be loaded and activated automatically at the start of every quiz attempt. It shall register event listeners that monitor the following browser-level behaviors: changes in page visibility indicating tab-switching or window minimization, window blur events indicating focus loss from the browser window, keydown events matching screenshot capture keyboard shortcuts on Windows and macOS, and clipboard copy and paste events within the quiz interface. All detected events shall trigger an immediate asynchronous POST request to the backend, which shall validate and store the event in the activity log with the event type, the attempt ID, and a precise timestamp. A violation counter shall be maintained on the client side. When the counter exceeds a defined warning threshold, a visible warning notification shall be displayed to the student. When it exceeds a termination threshold, the quiz shall be automatically submitted and a notice shall be displayed explaining the reason.

FR-11: Result Generation and Display

Immediately upon quiz submission whether submitted manually by the student or automatically by the timer the system shall calculate the student's total score by summing the marks awarded for all questions. It shall compute the percentage score relative to the total available marks. A result summary shall be displayed to the student showing the total marks obtained, the percentage, and a pass or fail indication based on a configurable passing threshold. A question-by-question breakdown shall also be displayed, showing the student's answer, the correct answer for objective questions, and the AI-assigned score for subjective questions. The result record shall simultaneously be stored in the database and made accessible to the teacher through the reporting dashboard.

FR-12: Teacher Reporting Dashboard

The system shall provide teachers with a dedicated reporting interface accessible after a quiz has closed. The dashboard shall display class-wide performance statistics for each quiz including the number of attempts, the average score, the highest and lowest scores, the pass rate, and the score distribution across defined ranges. Question-level analytics shall also be displayed, identifying which questions had the lowest average scores across all student attempts, which may indicate unclear wording or excessive difficulty. Individual student results shall be accessible through a sortable and searchable results table, and each student's detailed attempt breakdown shall be viewable upon selection.

FR-13: Administrative User Management

The system shall provide administrators with a user management panel displaying all registered accounts in a searchable and filterable table. Administrators shall be able to view account details, modify a user's assigned role, deactivate an account to prevent login without permanently deleting associated data, and reactivate previously deactivated accounts. Administrators shall also have access to a system wide activity log that records all quiz sessions and anti-cheating events, allowing oversight of the system's security monitoring output.

3.1.2 Non-Functional Requirements

NFR-01: Security All passwords shall be stored using bcrypt hashing with an appropriate salt factor. All client-server communication shall be conducted over HTTPS. JSON Web Tokens used for session management shall have a defined expiry period. Role-based middleware shall validate every request for protected resources before processing.

Non-functional requirements define the quality attributes and operational standards that the system must satisfy. While functional requirements describe what the system does, non-functional requirements describe how well it must do it. These requirements are equally important to the system's success, as they determine whether the platform is secure, reliable, and suitable for real-world educational use.

Non-Functional Requirements of the Quizly System

Req. ID	Category	Requirement Description
NFR-01	Security	Passwords hashed with bcrypt; all communication over HTTPS; JWT expiry enforced; role-based middleware on all protected routes
NFR-02	Performance	Quiz submission and grading returned within acceptable response time; NLP pipeline completes within a reasonable duration for standard documents
NFR-03	Scalability	Modular architecture supports horizontal scaling; database design accommodates growth in users, quizzes, and results without structural changes
NFR-04	Usability	Interface operable by non-technical users; all primary workflows completable without external documentation; error messages descriptive and actionable
NFR-05	Reliability	Data integrity maintained at all times; quiz session data preserved on connection interruption; database transactions prevent corruption from concurrent writes
NFR-06	Maintainability	Consistent coding conventions followed throughout; modular codebase structure allows individual component updates without affecting unrelated modules

Table 2.2: Non-functional Requirements

NFR-01: Security

Security is the most critical non-functional requirement for Quizly given that the system handles personal user data and is designed specifically to enforce academic integrity. All user passwords shall be stored using the bcrypt hashing algorithm with a salt round configuration that provides an appropriate balance between security strength and processing time. No plaintext passwords shall be stored or transmitted at any point. All communication between the client browser and the server shall be conducted over HTTPS, ensuring that data in transit is encrypted and protected from interception. User sessions shall be managed using JSON Web Tokens with a defined expiry period, after which re-authentication is required. Every API endpoint that serves protected resources shall be protected by authentication middleware that validates the token's signature and expiry before any data is processed or returned. Role-based access checks shall be applied as a second layer to enforce that authenticated users can only access resources appropriate to their assigned role. Input validation shall be applied on both client and server sides to prevent injection attacks and malformed data from reaching the database.

NFR-02: Performance

The system shall respond to user interactions within a time frame that supports a smooth and uninterrupted experience, particularly during quiz attempts where delays could cause anxiety or loss of progress. Quiz pages shall load promptly upon selection. Quiz submissions shall be processed and results returned without noticeable delay for standard quiz sizes. The AI grading computation for individual short-answer questions shall complete within an acceptable duration. The NLP document processing pipeline, which is the most computationally intensive operation in the system, shall complete within a reasonable time for documents of typical lecture note length, and the teacher shall receive clear feedback during processing to indicate that the system is working.

NFR-03: Scalability

The system architecture shall be designed in a modular fashion so that individual components can be scaled independently as usage grows. The frontend, backend, and database layers are physically separable, allowing each to be deployed and scaled on separate infrastructure if required. The database schema shall be designed so that increasing volumes of users, quizzes, and result records do not require structural changes to the tables. The MongoDB collections storing unstructured data shall similarly accommodate growth in document volume without schema migration. The backend API shall be stateless by design using JWT-based session management rather than server-side sessions which enables horizontal scaling through load balancing when multiple server instances are required.

NFR-04: Usability

The system shall be designed with non-technical end users as the primary audience. Teachers and students shall be able to complete all primary workflows without reference to any external documentation or training material. Navigation shall be logical, consistent, and clearly signposted throughout the application. Error messages shall be specific, readable, and actionable a user who encounters an error shall understand what went wrong and what they can do to resolve it. The quiz attempt interface shall be minimalist and free of distractions, allowing students to focus entirely on the assessment. All interactive elements shall respond to user actions in a manner consistent with common web application conventions.

NFR-05: Reliability

The system shall maintain data integrity at all times. Database operations involving multiple related records such as storing all responses for a quiz submission shall be executed as transactions so that either all records are committed successfully or none are, preventing partial data states. Quiz session data including the student's current responses shall be preserved in the event of a temporary network interruption, and the student shall be able to resume or submit the session without data loss where technically feasible. The system shall handle unexpected errors gracefully, returning informative error responses to the client rather than crashing or exposing internal system details.

NFR-06: Maintainability

The codebase shall follow consistent and documented coding conventions throughout both the frontend and backend. Functions and modules shall be clearly named to reflect their purpose. All non-obvious logic shall be accompanied by inline comments that explain the intent of the code. The system shall be organized into clearly separated modules authentication, quiz

management, NLP generation, AI grading, anti-cheating, and reporting such that changes to any one module do not require modifications to unrelated modules. This separation of concerns ensures that the system can be maintained, extended, and debugged efficiently as the project evolves.

3.2 Feasibility Study

Before proceeding with system design and implementation, a feasibility study was conducted to evaluate whether the proposed system is viable from technical, economic, operational, and legal perspectives. This analysis ensures that the project is grounded in realistic constraints and that risks related to resource availability, cost, usability, and compliance have been considered in advance.

Feasibility Study Summary

Feasibility Type	Assessment	Justification
Technical	Feasible	All required technologies are mature, well-documented, and within the development team's capability
Economic	Feasible	Entire technology stack is open-source; no licensing costs; free-tier deployment sufficient for development and testing phases
Operational	Feasible	Web-based interface requires no installation; manageable by non-technical users; single administrator sufficient for system operation
Legal	Feasible	System design complies with PECA 2016, ETO 2002, and constitutional privacy principles; data minimization practices applied throughout

Table 3.3: Feasibility Study

3.2.1 Technical Feasibility

All technologies required to build Quizly are mature, reliable, and supported by extensive documentation and active developer communities. React.js is a well-established industry standard for developing modern and responsive web interfaces and is widely used by leading technology companies. Node.js with Express.js provides a robust backend environment capable of handling application logic, user management, and API communication efficiently. MySQL is a highly reliable relational database management system that offers strong support for structured data storage, data integrity, indexing, and query optimization. It is widely adopted in both academic and industry projects due to its stability, performance, and ease of maintenance. The NLP and AI functionalities are implemented using readily available open-source libraries and tools that have been successfully utilized in similar educational technology solutions. The development team possesses the necessary technical knowledge and practical experience gained through their academic coursework to effectively work with these technologies. Based on the availability of resources, technical expertise, and successful implementation of similar systems, the proposed project is considered fully technically feasible.

3.2.2 Economic Feasibility

The entire technology stack used in this project is available under open-source or free-tier licensing, eliminating all software acquisition costs. Development labor is provided through the academic program and therefore does not represent a financial cost to the project. For hosting during the development and testing phases, free-tier or low-cost cloud infrastructure options are available that fully meet the scale requirements of this project. Looking beyond the academic context, if the system were to be adopted by an educational institution, the ongoing operational costs would be substantially lower than those associated with proprietary commercial assessment platforms, which typically carry per-seat licensing fees or usage-based charges. The project is economically feasible for both its development phase and its potential future institutional adoption.

3.2.3 Operational Feasibility

The system is designed with operational simplicity as a core design principle. The web-based interface requires no software installation on end-user devices and is accessible through any modern web browser. Teachers and students can learn to use the platform's core functions within a single orientation session. All primary workflows quiz creation, quiz attempt, and result viewing are designed to be intuitive and completable without technical expertise or external guidance. Administrative functions are straightforward and can be managed by a single designated individual without specialist IT knowledge. The operational demands of running the system are well within the capacity of a typical educational institution's existing IT infrastructure.

3.2.4 Legal Feasibility

The system collects and processes personally identifiable information including user names, email addresses, academic performance data, and behavioral event logs generated during quiz sessions. The legal implications of this data handling have been assessed in the context of applicable Pakistani law and general data protection principles. The Prevention of Electronic Crimes Act 2016 (PECA) establishes provisions relevant to the secure handling of digital data and unauthorized access to information systems. The Electronic Transactions Ordinance 2002 provides a legal framework for electronic records and digital authentication. The right to privacy under Article 14 of the Constitution of Pakistan is also relevant to the collection of behavioral monitoring data.

To ensure compliance with these frameworks, Quizly applies data minimization principles only the personal data strictly necessary for assessment functionality is collected. All stored data is protected through encryption and role-based access controls. Users are informed about the data collected and its purpose before they begin using the system. The anti-cheating monitoring features, which involve the logging of browser behavior events during quiz sessions, are disclosed to students through an explicit notice presented before each quiz attempt begins. The system does not collect biometric data or any personal information beyond what is necessary for its defined assessment functions. On the basis of this analysis, the project is legally feasible and designed with responsible data handling practices embedded throughout.

3.3 Use Case Diagram

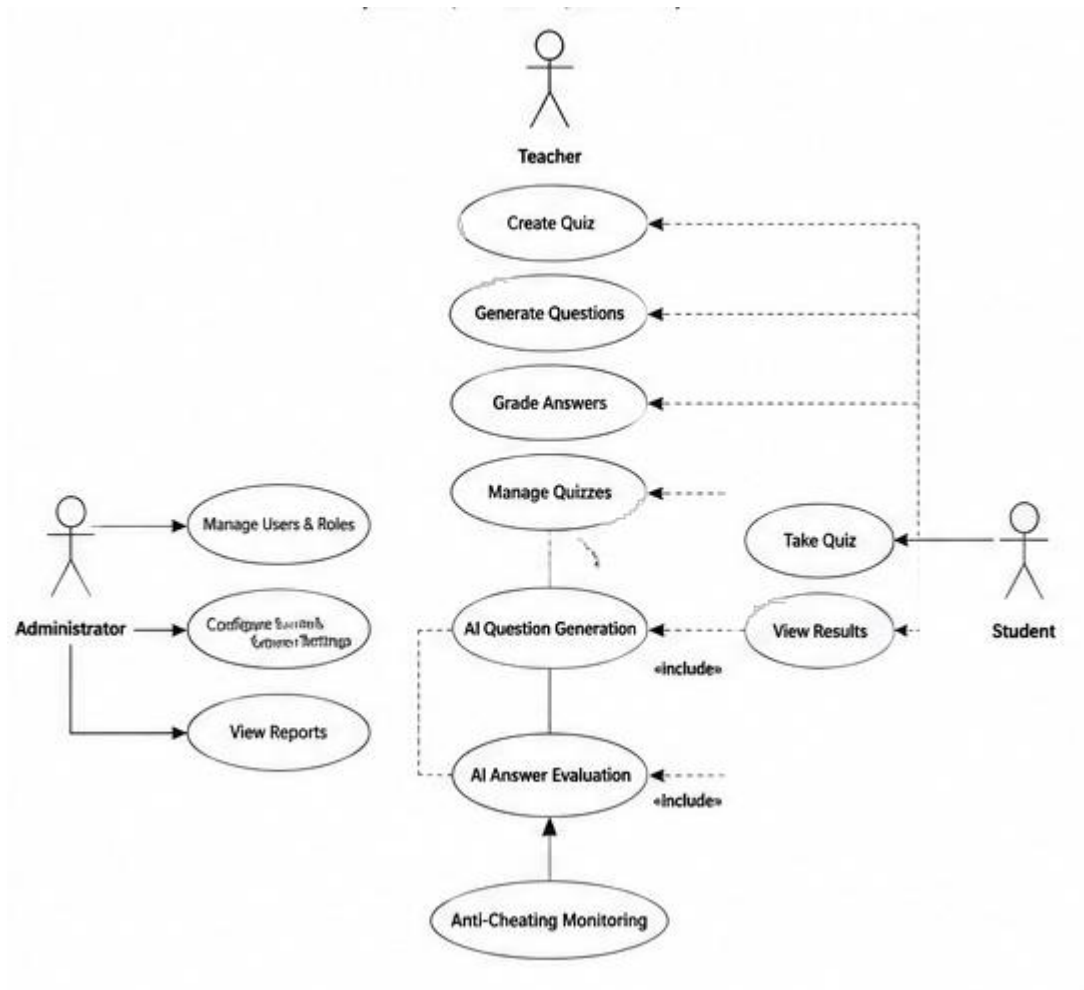


Figure 3.1: Use Case Diagram

Use Case Diagram — Quizly System (Admin, Teacher, Student)

The system involves three primary actors, each with a defined set of interactions:

Administrator: Manages the system at the configuration and user level. Use cases include managing user accounts, assigning and modifying user roles, viewing system activity logs, and monitoring flagged quiz sessions.

Teacher: Manages all assessment-related activities. Use cases include creating quizzes manually, uploading lecture material for automated question generation, reviewing and editing generated questions, publishing and scheduling quizzes, assigning quizzes to students, monitoring active quiz sessions, and viewing performance reports.

Student: Interacts with the system as an assessment participant. Use cases include viewing assigned quizzes, attempting a quiz within the secure monitored environment, submitting responses, and viewing quiz results.

All three actors must authenticate before accessing any system function. The anti-cheating module is included as a mandatory component in the quiz attempt use case.

3.4 Activity Diagrams

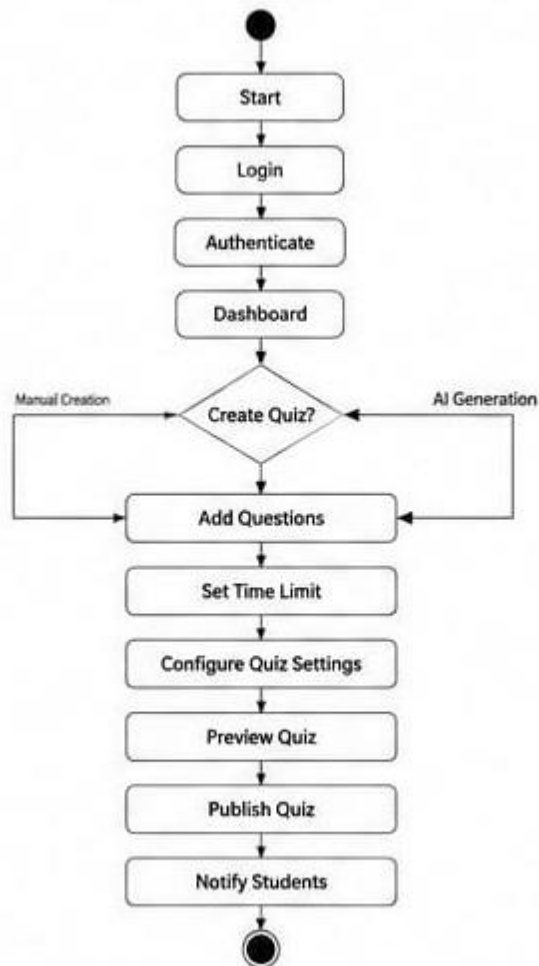


Figure 3.2: Teacher Activity Diagram

Activity Diagram — Teacher Workflow (Login to Publish Quiz)

Teacher Activity Flow: The teacher logs into the system and is directed to the dashboard. They choose to either create a quiz manually or generate one automatically. If choosing automated generation, they upload a document, wait for NLP processing to complete, and are presented with draft questions which they can edit and then publish. If creating manually, they enter questions one by one and configure quiz settings before publishing. Once a quiz is live, the teacher can monitor active sessions and, after the quiz closes, access the reporting dashboard to review results.

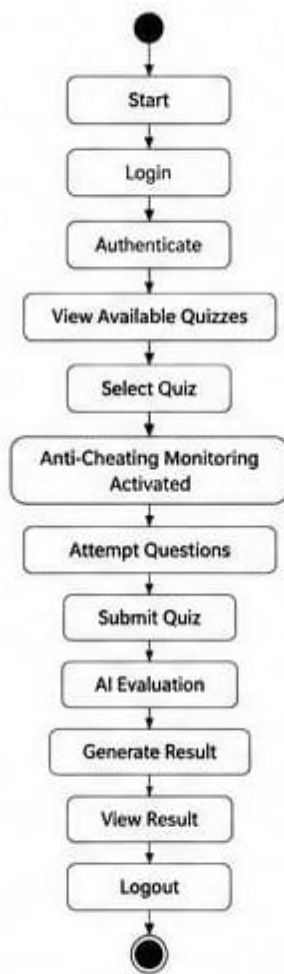


Figure 3.3: Student Activity Diagram

Activity Diagram — Student Workflow (Login to Results)

Student Activity Flow: The student logs into the system and views their list of assigned quizzes. They select an available quiz and are shown the quiz rules including the anti-cheating policy. Upon confirming, the quiz interface loads and the anti-cheating module activates. The student answers questions and submits the quiz before the time limit expires. The system grades the submission and displays the result. If any anti-cheating events were logged during the session, these are recorded for teacher review.

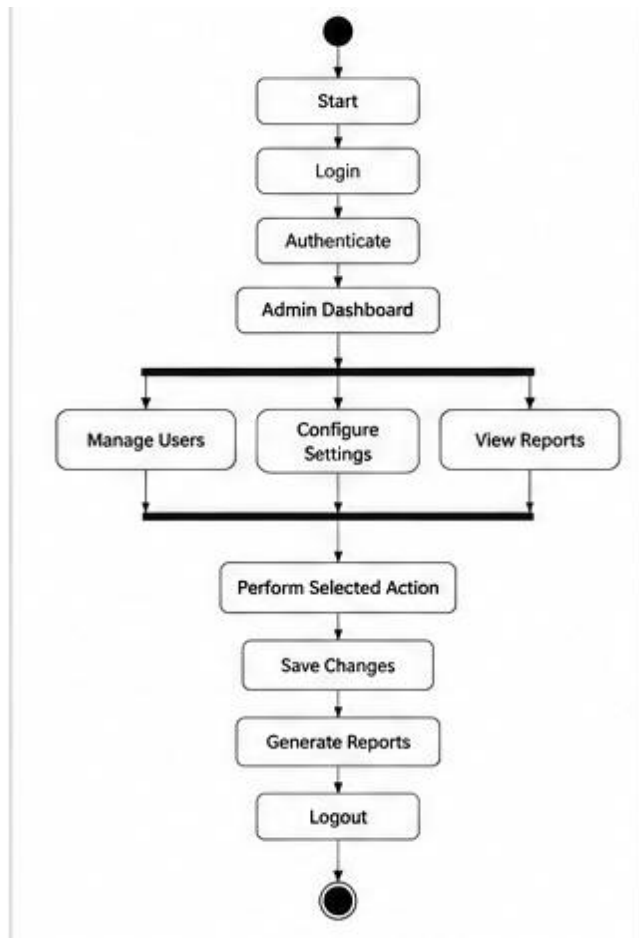


Figure 3.4: Administrator Diagram

Activity Diagram — Administrator Workflow

Administrator Activity Flow: The administrator logs in and accesses the administrative panel. They can view all registered users, modify account status or roles, and review the system activity log. If a flagged quiz session requires review, the administrator can access the detailed event log for that session and escalate it to the relevant teacher if necessary.

3.5 Sequence Diagrams

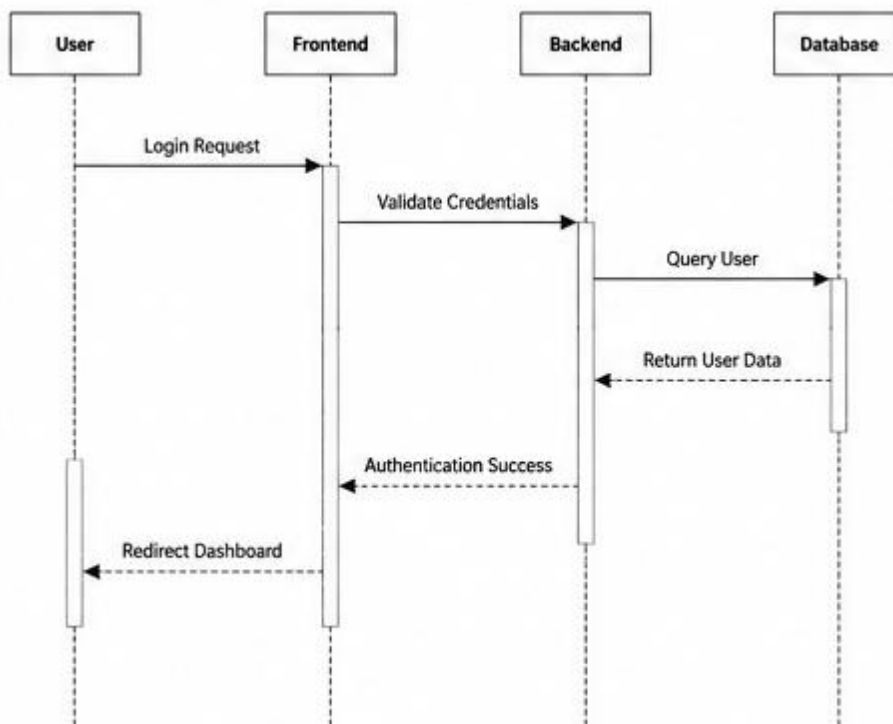


Figure 3.5: User Login Diagram

Sequence Diagram — User Login Process

Login Sequence: The user submits credentials through the frontend login form. The frontend sends an HTTP POST request to the authentication endpoint on the backend. The backend retrieves the user record from the database and compares the submitted password against the stored hash using bcrypt. If validation succeeds, the backend generates a JWT and returns it to the frontend. The frontend stores the token and redirects the user to their role-appropriate dashboard.

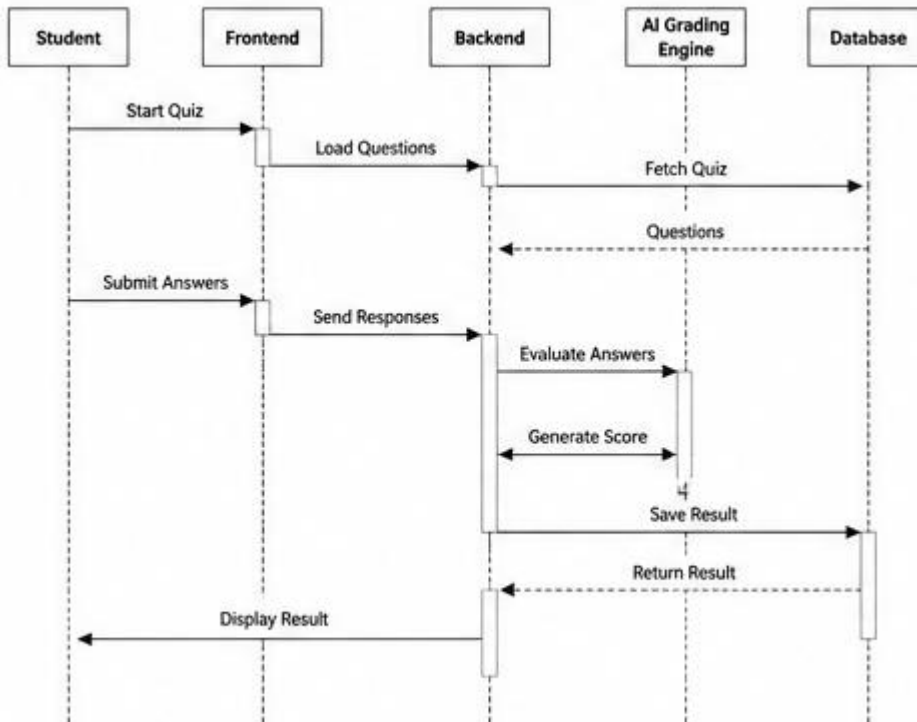


Figure 3.6: Quiz Attempt Diagram

Sequence Diagram — Quiz Attempt and AI Grading Process

Quiz Attempt Sequence: The student selects a quiz from the dashboard. The frontend sends a request to the backend to load the quiz data. The backend validates that the student has access to this quiz and that it is within its active time window. The quiz data is returned to the frontend and the interface is rendered. The anti-cheating module registers event listeners. As the student answers questions, responses are stored temporarily in the frontend state. Upon submission, responses are sent to the backend. The backend routes objective answers to the auto-grader and subjective answers to the AI grading module. Scores are aggregated, a Result record is created, and the final score is returned to the frontend for display.

AI Grading Sequence: The backend receives a student's short-answer response along with the associated model answer. Both texts are sent to the AI grading module. The module preprocesses both inputs, generates vector embeddings, computes cosine similarity, and returns a similarity score. The backend converts this score to a mark based on the question's total marks and stores the result.

3.6 Class Diagram

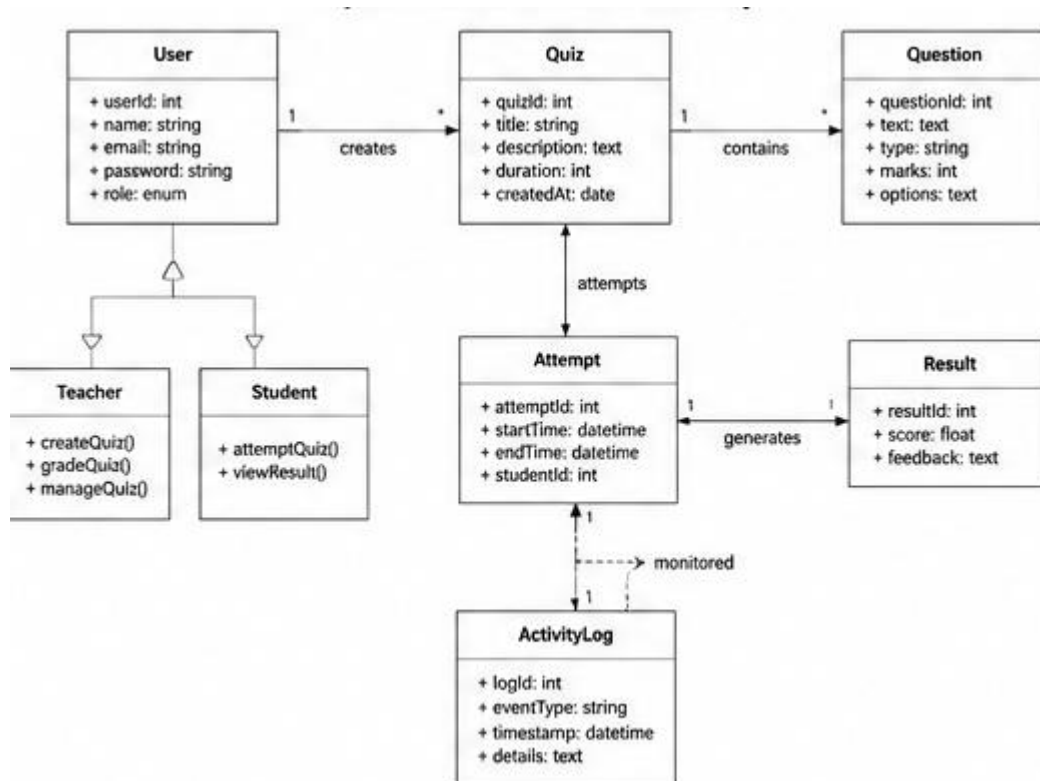


Figure 3.7: Class Diagram

Class Diagram — Object-Oriented Structure of Quizly

The system's object model includes the following primary classes and their relationships:

User (Abstract Base Class): Attributes include `userID`, `name`, `email`, `passwordHash`, `role`, and `accountStatus`. Provides common methods for authentication and profile management. Serves as the parent class for `Administrator`, `Teacher`, and `Student`.

Administrator: Extends `User`. Additional methods include `manageUsers()`, `viewActivityLogs()`, and `modifyRoles()`.

Teacher: Extends `User`. Additional attributes include `assignedCourses`. Methods include `createQuiz()`, `uploadMaterial()`, `viewResults()`, and `assignQuiz()`.

Student: Extends `User`. Additional attributes include `enrolledGroups`. Methods include `attemptQuiz()` and `viewResults()`.

Quiz: Attributes include `quizID`, `title`, `description`, `timeLimit`, `startTime`, `endTime`, `createdByTeacherID`, and `status`. Associated with one or more `Question` objects and one or more student assignments.

Question: Attributes include `questionID`, `quizID`, `questionText`, `questionType`, `marksAllocated`, and `modelAnswer`. For multiple-choice questions, additional attributes store the option set and the correct option identifier.

Response: Attributes include responseID, attemptID, questionID, studentAnswer, and assignedScore. Linked to both the Question and the quiz Attempt.

Attempt: Attributes include attemptID, quizID, studentID, startTime, endTime, totalScore, and percentageScore. Aggregates all Response objects for a single student's quiz session.

ActivityLog: Attributes include logID, attemptID, eventType, eventTimestamp, and additionalDetails. Records individual anti-cheating events during a quiz session.

3.7 Database Design

The database design for Quizly uses MySQL as the primary database management system for storing and managing all structured application data. MySQL was selected due to its reliability, strong relational capabilities, data integrity features, and suitability for handling user, quiz, assessment, and activity-related information efficiently.

Database Entities, Key Attributes, and Relationships

Entity	Database	Key Attributes	Relationships
Users	MySQL	userID, fullName, email, passwordHash, role, accountStatus	Referenced by quizzes, attempts, and activity logs
Quizzes	MySQL	quizID, createdByTeacherID, title, timeLimit, startTime, endTime, status	Linked to questions and quiz assignments
Questions	MySQL	questionID, quizID, questionText, questionType, marksAllocated, modelAnswer	Belongs to a specific quiz
Quiz_Assignments	MySQL	assignmentID, quizID, studentID	Links quizzes with assigned students
Attempts	MySQL	attemptID, quizID, studentID, startTime, endTime, totalScore, percentageScore	Stores student quiz attempts
Responses	MySQL	responseID, attemptID, questionID, studentAnswer, assignedScore	Linked to attempts and questions
Activity_Logs	MySQL	logID, attemptID, eventType, eventTimestamp	Stores anti-cheating and user activity events
NLP_Documents	MySQL	documentID, teacherID, rawText, extractedContent, uploadDate	Stores uploaded documents used for question generation
Generated_Questions	MySQL	generatedQuestionID, documentID, questionText, modelAnswer, reviewStatus	Stores AI-generated draft questions before publishing

Table 3.4: Entites,Keys & Realltionships

3.8 System Architecture

The system follows a five-layer architecture designed for clarity, separation of concerns, and scalability.

Layer 1 — Presentation Layer (Frontend): Built using React.js, this layer is responsible for all user-facing interactions. It implements component-based UI rendering for each user role's interface, handles client-side validation, manages quiz navigation and timing, and communicates with the backend exclusively through REST API calls over HTTPS. The anti-cheating monitoring script is loaded and activated at this layer when a quiz begins.

Layer 2 — Business Logic Layer (Backend): Implemented using Node.js with Express.js, this layer handles all application logic and serves as the coordinator between all other layers. It manages authentication and authorization, processes API requests from the frontend, enforces role-based access rules through middleware, orchestrates quiz creation and retrieval, routes grading requests, and manages session data. All business rules are implemented at this layer, keeping the presentation and data layers clean and independent.

Layer 3 — AI and NLP Layer: This layer contains the intelligent processing components. The NLP sub-module accepts preprocessed document text, performs tokenization, sentence scoring, keyword extraction, and question formulation to produce candidate quiz questions. The AI Grading sub-module accepts student responses and model answers, generates semantic embeddings, computes similarity scores, and returns numerical grades. These components are designed as services that can be called by the business logic layer without direct coupling to any other system component.

Layer 4 — Database Layer: This layer manages all persistent data storage. MySQL handles structured relational data where referential integrity is critical, including users, quizzes, questions, attempts, results, generated questions, and activity logs. The database is designed to efficiently support both transactional operations and data retrieval processes. The business logic layer interacts with the database through abstracted data access logic that isolates the rest of the system from database-specific implementation details.

Layer 5 — Security and Monitoring Layer: This layer operates across both the frontend and backend and is responsible for enforcing academic integrity and system security. On the frontend, JavaScript event listeners monitor browser-level activity during quiz sessions. On the backend, all API endpoints are protected through JWT validation middleware, role-based access checks are applied before any data is served, and all anti-cheating events received from the frontend are validated and stored. Secure password storage, HTTPS communication, and input validation are enforced throughout.

3.9 User Interface Design

The user interface is designed around the principle of role-appropriate simplicity. Each user type sees only the features and information relevant to their function, reducing cognitive load and minimizing the risk of errors.

Login Page: A clean and user-friendly authentication interface designed for secure user access. The screen provides dedicated input fields for user credentials along with clear navigation options, enabling users to log in and access the system efficiently.

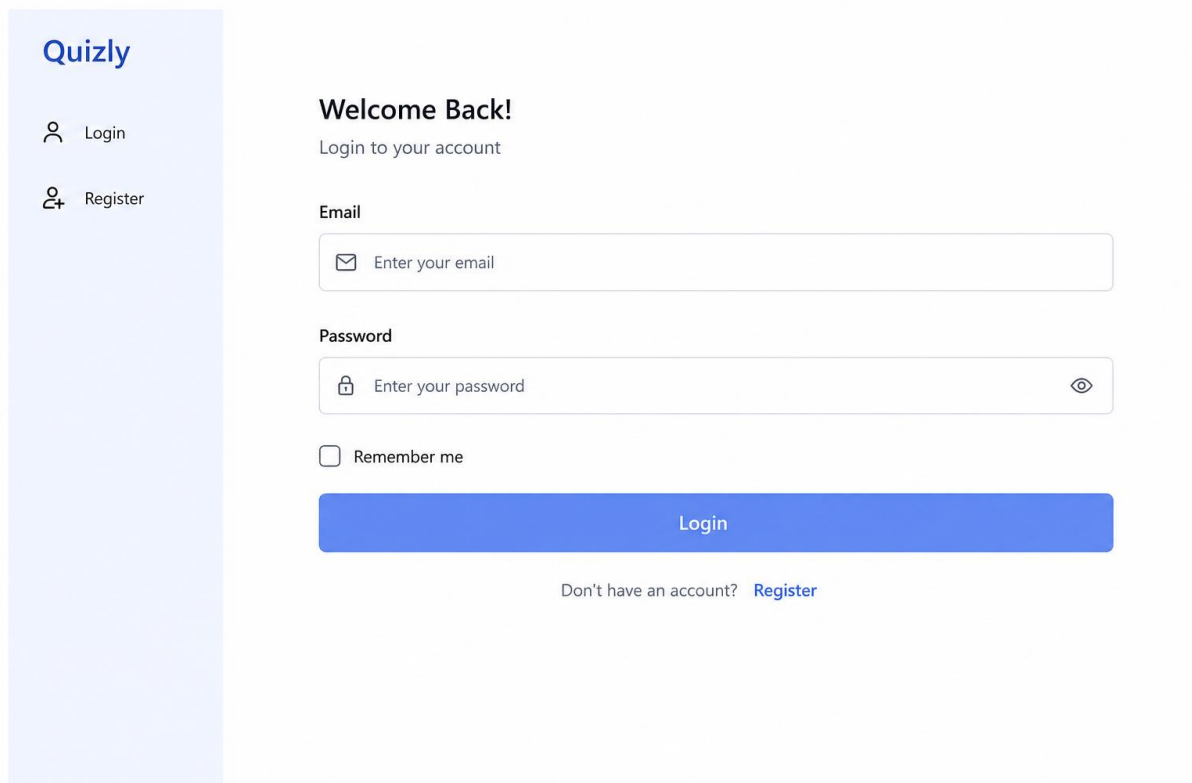


Figure 3.8: Login Page

(a) Wireframe — Login and Registration Page

Teacher Dashboard: An A clean dashboard providing teachers with an overview of quiz-related activities. The interface includes summary cards and a dedicated section for managing recently created quizzes, offering quick access to essential functions

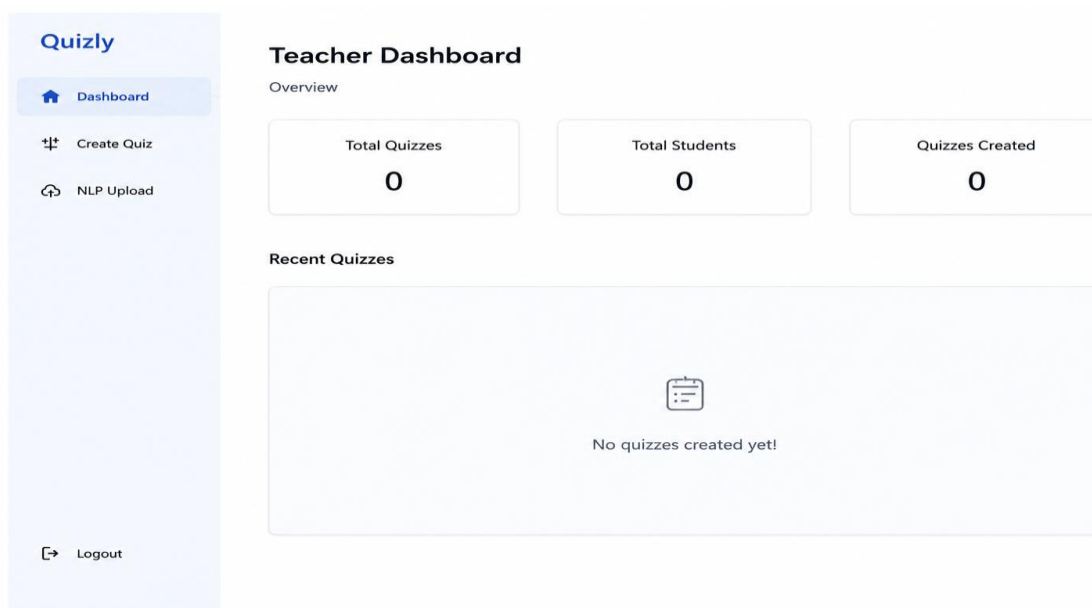
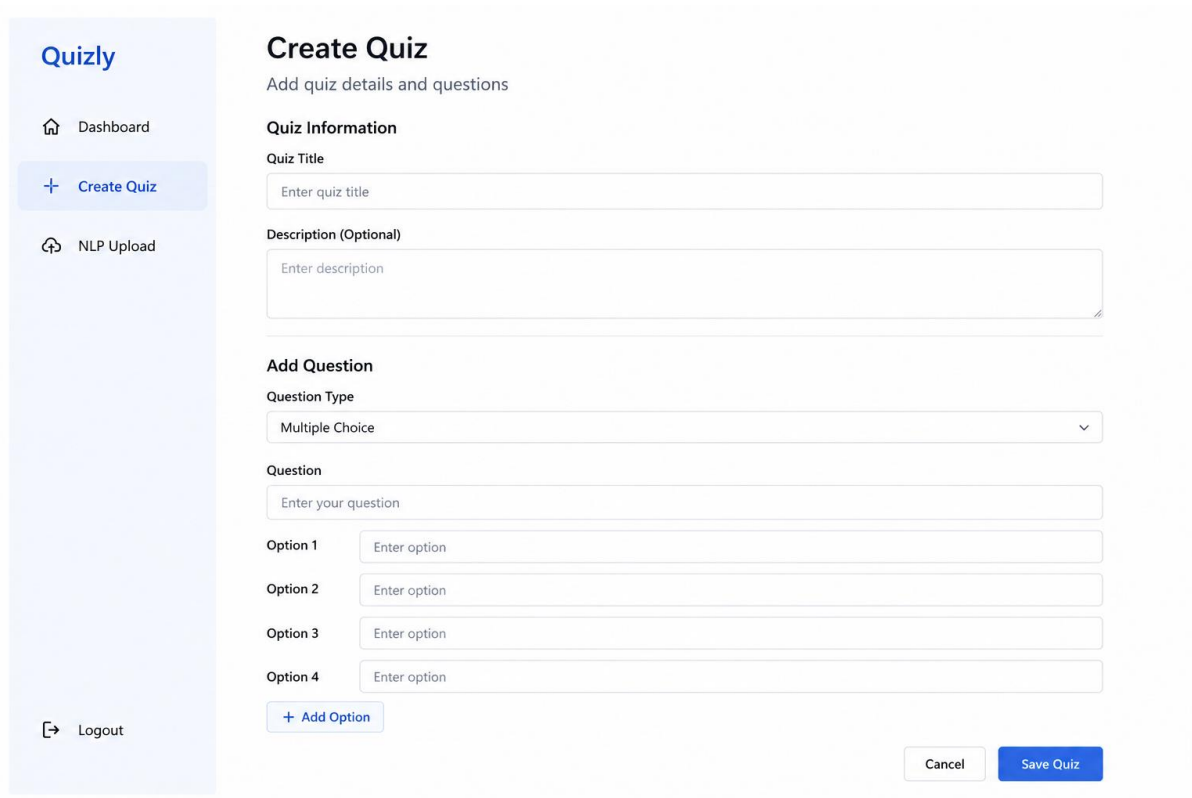


Figure 3.9: Teacher Dashboard

(b) Wireframe — Teacher Dashboard

Quiz Creation Interface: A simple form-based interface that allows teachers to create quizzes by entering quiz details, adding questions, and defining multiple-choice options. The layout supports easy quiz management with clear input fields and action buttons.



The wireframe shows a 'Quizly' teacher dashboard with a sidebar and a main content area. The sidebar contains links for 'Dashboard', 'Create Quiz' (highlighted), 'NLP Upload', and 'Logout'. The main content area is titled 'Create Quiz' with a subtitle 'Add quiz details and questions'. It is divided into two sections: 'Quiz Information' and 'Add Question'. The 'Quiz Information' section has a 'Quiz Title' text input field and a 'Description (Optional)' text area. The 'Add Question' section has a 'Question Type' dropdown menu set to 'Multiple Choice', a 'Question' text input field, and four 'Option' text input fields labeled 'Option 1' through 'Option 4'. There is an '+ Add Option' button below the options. At the bottom right of the form are 'Cancel' and 'Save Quiz' buttons.

Figure 3.10: Quiz Creation

(c) Wireframe — Quiz Creation Interface (Manual Mode)

NLP Upload Interface: A streamlined interface for uploading study materials and generating quiz questions automatically. It includes a document upload area, configurable generation settings, and a straightforward action button for initiating the process.

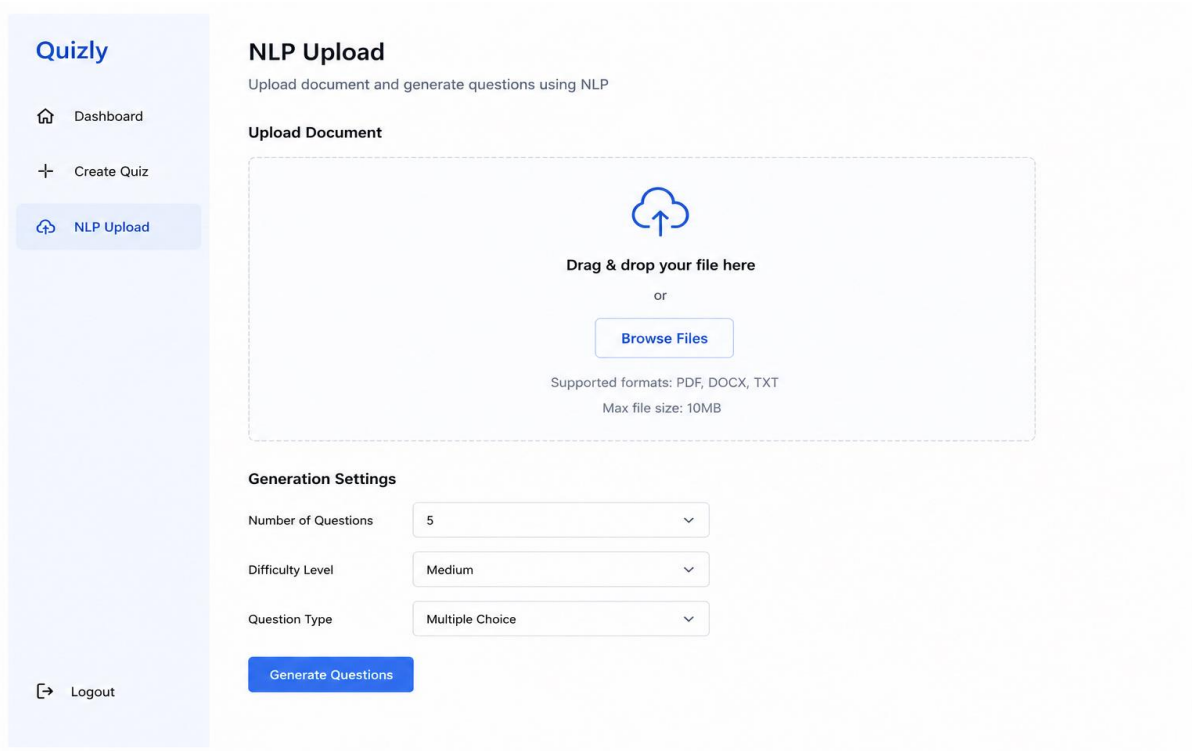


Figure 3.11: NLP Interface

(d) Wireframe — NLP Document Upload and Question Review

Student Quiz Interface: A user-friendly dashboard that presents students with an overview of their quiz activities and progress. It provides dedicated sections for assigned quizzes and recent results in a simple, organized layout.

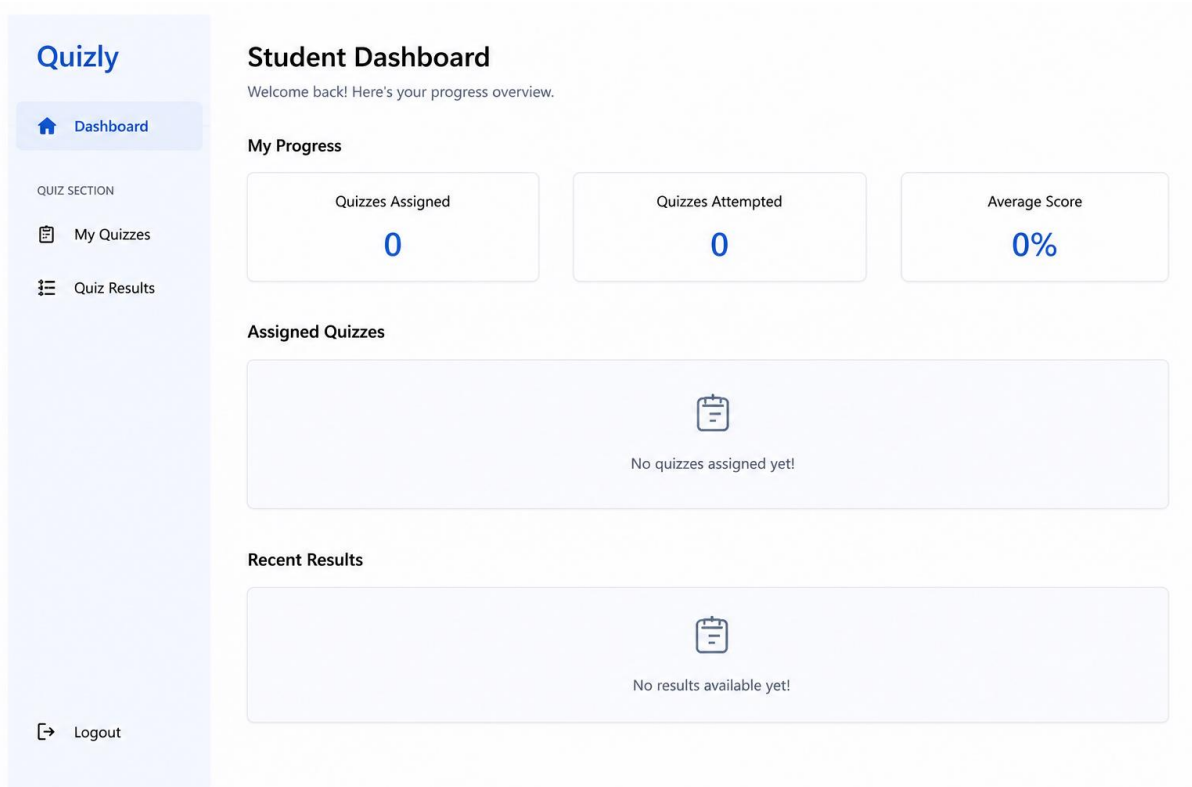


Figure 3.12: Student Dashboard

(e) Wireframe — Student Dashboard

Quiz Attempt Interface: A structured quiz-taking interface displaying the current question, answer options, and navigation controls. A timer and basic security notifications are included to support a controlled assessment environment.

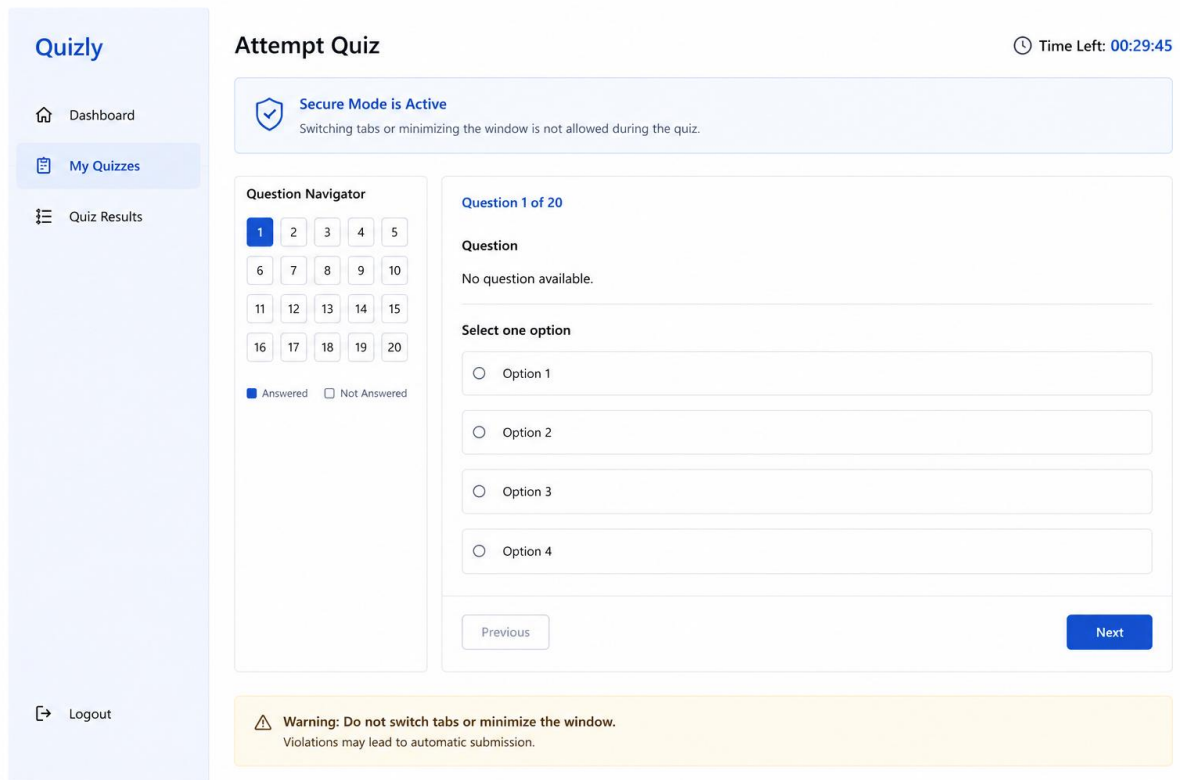


Figure 3.13: Quiz Attempt Interface

(f) Wireframe — Quiz Attempt Interface

Results and Analytics : A clean results page that summarizes quiz performance and displays question-wise outcomes. The interface also provides a dedicated section for feedback while maintaining a simple and easy-to-read layout.

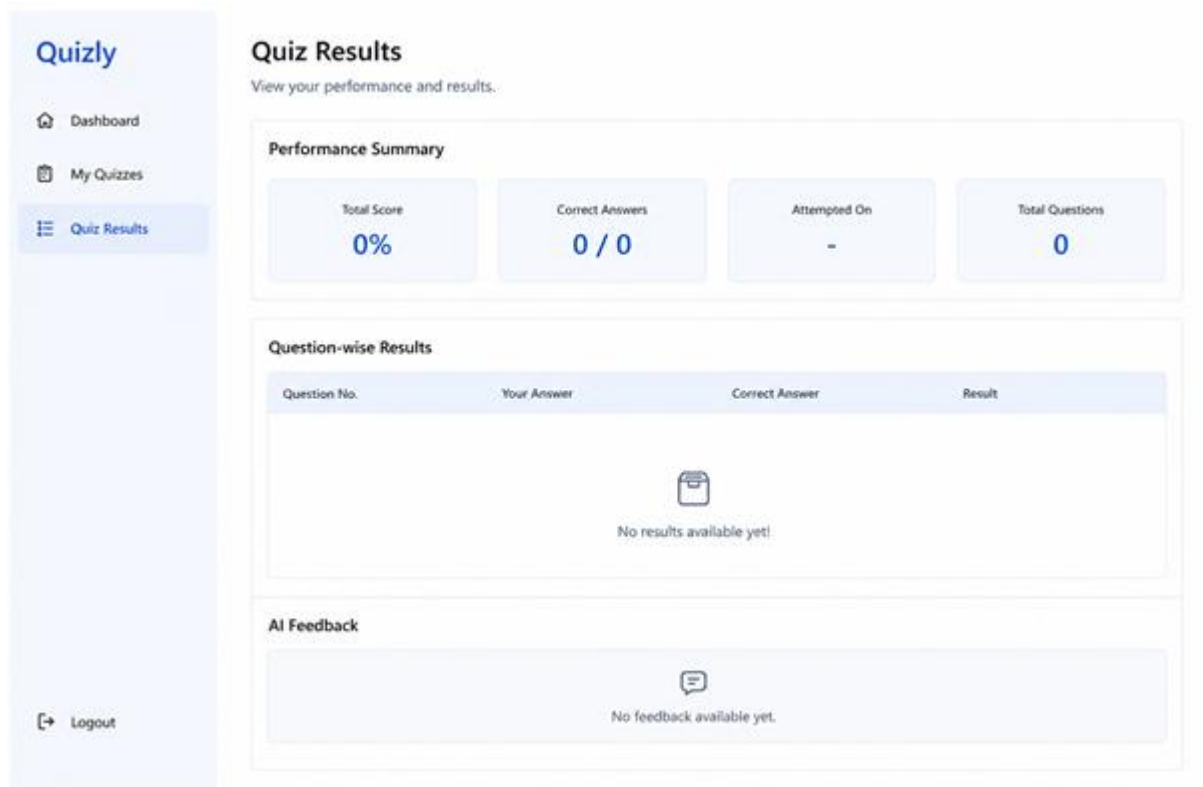


Figure 3.14: Quiz Result

(g) Wireframe — Results and Analytics Page

Admin Panel: A centralized administration interface for managing system users and resources. The layout includes a user management table and basic administrative controls, enabling efficient oversight of the platform.

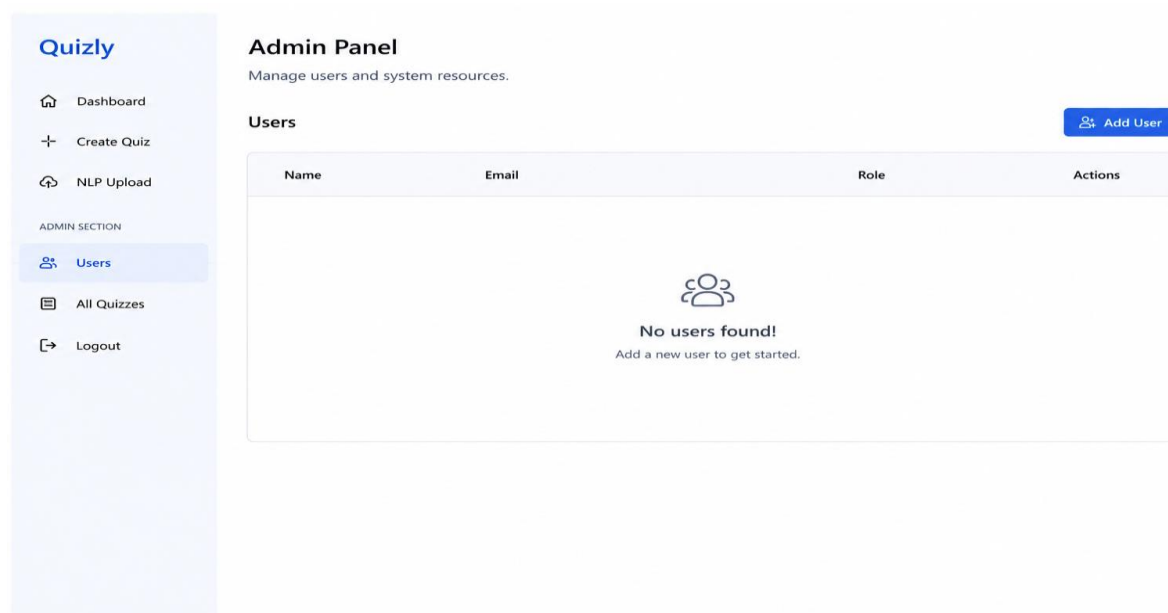


Figure 3.15: Admin Panel

(h) Wireframe — Administrator Panel

3.10 SDLC Model — Agile Methodology

The development of Quizly follows the **Agile Software Development Life Cycle**. Agile was selected as the most suitable methodology for this project because of the iterative and exploratory nature of building AI and NLP components, where system behavior cannot be fully predicted in advance and requires repeated testing and refinement throughout the development process.

Unlike the Waterfall model, which requires all requirements and design decisions to be finalized before any implementation begins, Agile allows for continuous adjustment as the project progresses. This flexibility was particularly important for Quizly because the accuracy of the AI grading module and the effectiveness of the anti-cheating detection mechanisms could only be properly evaluated through working software not through theoretical design alone. Each time a module was built and tested, new insights emerged that informed the next phase of development.

Sprint Structure

The project is organized into the following sprint phases:

- Sprint 1 (Research and Planning): Problem identification, literature review, comparative analysis, and project proposal.
- Sprint 2 (Requirements and Design): Functional and non-functional requirements definition, system architecture design, UML diagrams, ERD, and UI wireframes.
- Sprint 3 (Core Backend Development): User authentication module, database schema implementation, quiz management APIs.
- Sprint 4 (AI/NLP Integration): NLP-based question generation engine, AI grading module, and integration testing.
- Sprint 5 (Frontend Development): React.js UI implementation across all role-based dashboards and quiz interfaces.
- Sprint 6 (Anti-Cheating and Security): Implementation of tab-switch detection, screenshot blocking, clipboard restriction, and activity logging.
- Sprint 7 (Testing and QA): Unit, integration, system, and user acceptance testing. Bug fixing and performance optimization.
- Sprint 8 (Documentation and Deployment): Final documentation, user manual preparation, GitHub repository finalization, and system deployment.

Agile also provides a formal mechanism for managing scope. If development of any sprint's features takes longer than planned, lower-priority items can be moved to a later sprint without derailing the overall project. This scope management capability, combined with regular supervisor reviews, ensures that the project remains on track and that the most critical features are always prioritized.

3.11 Project Timeline

The project timeline spans the full duration of FYP-I and FYP-II, with clearly defined phases that map onto the Agile sprint structure described in the previous section. The below table

summarizes the major project phases, their key activities, and the planned timeline for completion.

Project Phases, Descriptions, and Timelines

Phase	Description	Timeline
Idea Approval and Proposal	Project concept finalized and documented; formal proposal submitted to supervisor for approval	February 2026
Requirement Analysis	Problem domain studied; existing systems reviewed; functional and non-functional requirements identified and finalized	February 2026
System Design	Architecture designed; all UML diagrams produced; database schema finalized; UI wireframes developed; AI and security design decisions documented	February – March 2026
Sprint 1 — Foundation	Development environment configured; database schema implemented; authentication and user management module developed and unit tested	April 2026, Week 1
Sprint 2 — Quiz Management	Manual quiz creation, quiz scheduling, and assignment module developed and tested	April 2026, Week 2
Sprint 3 — NLP Module	Document upload pipeline and NLP question generation module developed and integrated with quiz creation interface	April 2026, Week 3
Sprint 4 — AI Grading	AI grading engine developed; integrated with quiz submission workflow; grading accuracy validated	April 2026, Week 4
Sprint 5 — Anti-Cheating	Anti-cheating monitoring module developed; integrated with quiz attempt interface; event logging verified	May 2026, Week 1
Sprint 6 — Reporting and Admin	Reporting dashboard, result display, and administrator panel developed and tested	May 2026, Week 2
Sprint 7 — Testing and Docs	Full testing cycle (unit, integration, system, UAT) conducted; defects resolved; project report finalized	May 2026, Week 3–4
Final Presentation	Completed system demonstrated to internal and external examiners; final thesis submitted	June – July 2026

Table 3.5: Project Phases

Gantt Chart

The Gantt chart illustrates the planned schedule and distribution of project activities throughout the development lifecycle of Quizly. The project begins with requirement gathering, literature review, and system design, followed by development of the frontend, backend, AI-based modules, and anti-cheating mechanisms.

Testing, system integration, and validation activities are conducted during the later phases to ensure reliability, security, and performance of the application. Finally, project documentation, deployment, and final submission are completed in June – July 2026, ensuring timely achievement of all FYP objectives.

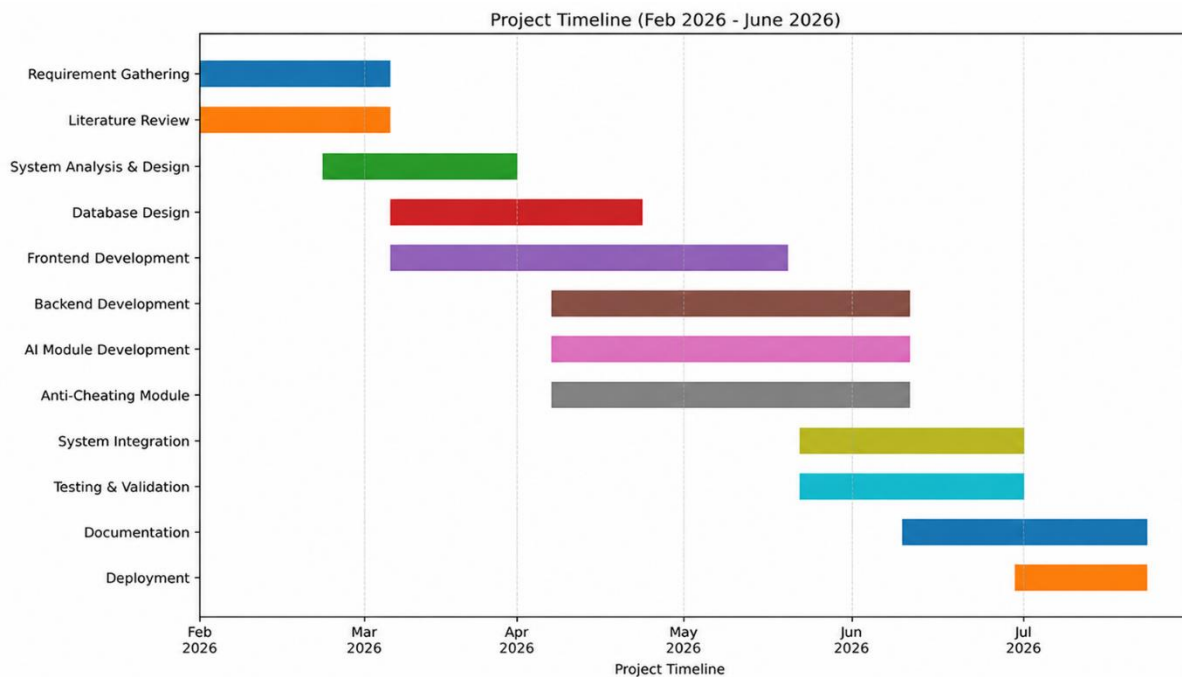


Figure 3.16: Gantt Chart

Gantt Chart — Project Timeline (Feb 2026 – June 2026)

3.12 Risk Analysis

Risk analysis was conducted at the project planning stage to identify potential challenges that could affect the timeline, quality, or integrity of the system. Each identified risk is assessed for its probability of occurrence and its potential impact on the project, and a specific mitigation strategy is defined. This analysis is revisited at the start of each sprint so that emerging risks can be identified and addressed promptly.

Risk Analysis Matrix

Risk	Probability	Impact	Mitigation Strategy
AI Grading Bias — model produces inaccurate or inconsistent scores for diverse student responses	Medium	High	Fine-tune model using diverse test datasets; validate AI scores against manual grading benchmarks during Sprint 7 testing; implement teacher score override functionality
Security Bypass — students identify methods to circumvent browser-level anti-cheating controls	Low	High	Implement multiple overlapping detection layers; conduct targeted security testing during Sprint 5; maintain comprehensive activity logs for post-session behavioral review
NLP Quality Issues — generated questions poorly formed or irrelevant for technical documents	Medium	Medium	Require mandatory teacher review before any generated question is published; iteratively improve NLP preprocessing based on testing feedback during Sprint 3

Time Constraints — semester duration insufficient for complete feature delivery	Medium	Medium	Apply Agile sprint planning with explicit prioritization; core features assigned to early sprints; scope adjustment decisions made in consultation with supervisor
Server Downtime — hosting infrastructure unavailable during a scheduled quiz window	Low	High	Select reliable hosting; implement client-side response preservation; establish student communication procedure for unplanned downtime
Data Loss — database failure occurs mid-submission during an active quiz attempt	Low	High	Wrap all quiz submission operations in database transactions; preserve frontend session state until server confirmation is received
Scope Creep — additional features requested during development expand scope beyond manageable limits	Medium	Medium	Maintain formal feature backlog; all scope additions reviewed against timeline impact before acceptance; new features deferred to future work if timeline is at risk

Table 3.6: Risk Analysis Matrix

AI Grading Bias

The AI grading engine uses semantic similarity models to evaluate student responses against model answers. These models are trained on general-purpose language data and may not perform with equal accuracy across all academic subject domains, particularly in highly technical fields where correct answers may use specialized terminology that the model does not associate strongly with the model answer's phrasing. A student who provides a technically correct response in non-standard wording may receive a lower score than deserved, which directly undermines the fairness of the assessment.

This risk is assessed as medium probability because the model performs well for general academic language but has not been fine-tuned on educational content specific to software engineering or related disciplines. The impact is high because grading inaccuracy is a core failure mode for the system's primary purpose. Mitigation involves systematic comparison of AI-assigned scores against manually assigned grades for the same responses during Sprint 7 testing, identification of consistent divergence patterns, and iterative refinement of the preprocessing and similarity scoring parameters. A teacher override function is also implemented, allowing teachers to manually adjust any AI-assigned score that they assess as incorrect, ensuring that the final grade always reflects a human-reviewed evaluation.

Security Bypass

The anti-cheating module operates entirely through JavaScript event listeners registered in the student's browser. While this approach is effective against the most common forms of dishonest behavior tab-switching, clipboard use, and screenshot keyboard shortcuts it has inherent limitations. A technically knowledgeable student could potentially use browser developer tools to inspect and disable specific event listeners, use a second physical device to photograph the screen, or employ operating system-level screenshot tools on some platforms that do not trigger browser keyboard events.

The probability of this risk is assessed as low because these circumvention methods require deliberate technical effort that most students would not undertake, and because attempting them during an examination carries its own risk of detection through behavioral patterns in the activity log. The impact is high because any successful undetected bypass creates an unfair advantage. Mitigation involves implementing multiple overlapping detection mechanisms tab-switch detection, window blur detection, keyboard shortcut blocking, and clipboard restriction so that circumventing any one layer does not defeat the entire monitoring system. Comprehensive activity logging also provides a post-session audit trail that can identify suspicious patterns even when real-time detection was incomplete.

NLP Quality Issues

The NLP question generation pipeline performs well when processing clearly written, well-structured academic prose. However, lecture notes in technical disciplines such as software engineering frequently contain code blocks, command-line instructions, mathematical expressions, abbreviated bullet points, and other content formats that the NLP module's sentence scoring and question generation patterns are not designed to handle effectively. When such content is processed, the generated questions may be incomplete, syntactically awkward, or semantically unclear.

The probability of this risk is medium because the target users software engineering students and teachers regularly work with precisely this type of technical content. The impact is medium rather than high because the mandatory teacher review step provides a quality gate that prevents any poorly formed question from reaching students without human approval. The teacher can edit, rephrase, or discard any generated question that does not meet acceptable standards. Mitigation therefore focuses on iterative improvement of the NLP preprocessing pipeline based on specific failure examples identified during Sprint 3 development and Sprint 7 testing, with the goal of reducing the proportion of generated questions that require significant editing.

Time Constraints

Developing a system that integrates seven distinct modules authentication, quiz management, NLP generation, AI grading, anti-cheating monitoring, reporting, and administration within a single academic semester represents a substantial workload. Unexpected complexity in any module, particularly the AI and NLP components which involve less predictable behavior than standard CRUD operations, could compress the time available for testing and documentation at the end of the project.

The probability is medium because complexity surprises are a normal part of software development, and the impact is also medium because the Agile sprint structure provides a built-in response mechanism. If a sprint's development takes longer than planned, the lowest-priority items from that sprint's backlog are moved to the next sprint rather than being rushed, preserving quality at the cost of scope. If timeline pressure persists, features designated as lower priority such as advanced analytics or additional anti-cheating layers are formally deferred to future work rather than compromising the delivery of core functionality. All scope adjustment decisions are made transparently and documented in consultation with the project supervisor.

Scope Creep

As development progresses and stakeholders including the supervisor and UAT participants interact with working versions of the system, there is a natural tendency for new feature ideas and enhancement requests to emerge. If these are incorporated into the development plan without formal evaluation of their impact on the timeline, the project scope can expand to the point where the original deliverables are at risk.

The probability is medium because feedback from supervisor reviews and UAT sessions is expected to generate improvement suggestions. The impact is medium because scope creep rarely causes immediate project failure but gradually erodes schedule margins until late-project pressure becomes severe. Mitigation involves maintaining a formal feature backlog where all new requests are recorded and assessed against the remaining timeline before any commitment to implementation is made. Requests that cannot be accommodated within the current project timeline are explicitly deferred to the future enhancements section of the project report, ensuring they are documented and acknowledged without creating unmanageable scope commitments.

Chapter 4: Implementation

4.1 Development Environment

The development of Quizly was conducted in a structured environment with the following specifications:

Operating System: Windows 11 on primary development machines.

Code Editor: Visual Studio Code, selected for its comprehensive JavaScript and React support, integrated terminal, built-in Git integration, and availability of essential extensions including ESLint for code quality enforcement, Prettier for consistent formatting, and the React Developer Tools extension for component inspection.

Version Control: GitHub was used for source code management and team collaboration. A shared repository with a branching strategy allowed both developers to work on separate modules concurrently and merge changes through pull requests, minimizing conflicts and maintaining a clean commit history.

API Testing: Postman was used throughout backend development to test and validate every API endpoint before frontend integration. It allowed the team to simulate all HTTP request types, inspect response structures, and test authentication flows without requiring a fully operational frontend.

Package Management: Node Package Manager (npm) was used to manage all dependencies for both the frontend and backend. A consistent lock file was maintained to ensure that both developers worked with identical dependency versions.

Local Development Servers: The React development server provided hot-reloading for frontend development. The Node.js backend was run locally during development with automatic restart on file changes managed by nodemon.

Database Tools: MySQL Workbench was used for designing and managing the MySQL schema during development and testing. It provided tools for database modeling, query execution, relationship management, and data verification, helping ensure the accuracy and consistency of the system's data storage layer.

4.2 Technologies Used

The technologies selected for this project were chosen based on their suitability for the specific requirements of each system layer, their maturity and community support, and their compatibility with the development team's existing knowledge base. The following table summarizes the complete technology stack, followed by a detailed justification for each selection.

Technologies Used

Technology	Category	Purpose
React.js	Frontend Framework	Component-based UI; state management; API integration via Axios
Node.js	Runtime Environment	Server-side JavaScript execution; full-stack language consistency
Express.js	Backend Framework	REST API routing; middleware pipeline; HTTP lifecycle management
MySQL	Relational Database	Structured data with referential integrity — users, quizzes, results
NLP.js / AI APIs	AI and NLP	Automated question generation; semantic similarity scoring for grading
bcrypt	Password Security	Password hashing with configurable salt rounds before database storage
Visual Studio Code	Development Tool	JavaScript and React development; integrated terminal; Git support
GitHub	Version Control	Collaborative development; branching; pull request workflow
Postman	API Testing	Backend endpoint testing and validation during development

Table 4.1: Used Technologies

React.js was selected for the frontend because its component-based architecture allows the interface to be built as a collection of reusable, independently testable UI elements. React's virtual DOM provides efficient rendering performance, which is important for the quiz interface where the timer and question navigation need to update frequently without causing visible lag. React's ecosystem also provides well-supported libraries for routing and state management that simplify the implementation of a multi-page, role-differentiated application.

Node.js with Express.js was selected for the backend because it allows the same JavaScript language to be used across both the frontend and backend, reducing the cognitive overhead of context-switching between languages during development. Node.js's non-blocking, event-driven architecture handles concurrent requests efficiently, which is important for a system where multiple students may be submitting quizzes simultaneously. Express.js provides a clean and minimal framework for defining API routes and applying middleware, keeping the backend codebase organized and maintainable.

MySQL was selected for structured relational data because the core entities in this system users, quizzes, questions, attempts, and results have clear and well-defined relationships that benefit from the referential integrity enforcement and ACID transaction guarantees that relational databases provide. MySQL is mature, well-documented, and widely supported in production environments.

GitHub was selected for version control and source code management because it provides a reliable platform for tracking project changes throughout the development lifecycle. It enables developers to maintain code history, manage different versions of the application, and collaborate efficiently through commits and repository management. The platform also serves as a secure backup mechanism, ensuring that project files remain accessible and recoverable during development and deployment activities.

4.3 System Modules

4.3.1 Authentication Module

The authentication module manages the complete user identity lifecycle. During registration, submitted form data is validated on both the client and server sides. The password is hashed using bcrypt with a salt round configuration appropriate for balancing security with performance. The hashed credential and user information are stored in the MySQL users table. During login, the submitted password is compared against the stored hash using bcrypt's comparison function, which is designed to be resistant to timing attacks. Upon successful authentication, a JWT is generated containing the user's ID and role as payload claims. This token is returned to the client, where it is stored in memory and attached as a Bearer token in the Authorization header of all subsequent API requests. Backend middleware validates the token's signature and expiry on every protected route before processing the request.

4.3.2 Quiz Management Module

The quiz management module provides the complete interface between teacher actions and quiz data. It handles quiz creation, editing, scheduling, and deletion. When a teacher creates a quiz, the module validates all required fields, creates the quiz record in MySQL, and creates associated question records linked to the quiz ID. The module also manages quiz assignments, creating records in the quiz assignments junction table that link specific quizzes to specific student accounts. Retrieval logic in this module filters quizzes based on the requesting user's role: teachers see all quizzes they have created, while students see only quizzes assigned to them that are within their active time window.

4.3.3 NLP Quiz Generation Module

The NLP quiz generation module accepts document text extracted from teacher uploads and processes it through a multi-stage pipeline. In the preprocessing stage, the text is cleaned, normalized, and tokenized into individual sentences. In the sentence scoring stage, each sentence is evaluated based on criteria including its length, position within the document, keyword density, and presence of named entities or factual constructs. The highest-scoring sentences are selected as candidate source sentences for question generation. In the question formulation stage, the module applies generation patterns to transform declarative sentences into interrogative structures, creating draft questions with associated model answers derived from the source sentence. The generated questions are stored as documents in the MongoDB generated questions collection and returned to the teacher interface as an editable list.

4.3.4 AI Grading Module

The AI grading module handles the evaluation of student responses to short-answer questions. When a submission containing subjective responses is received, each response is extracted along with its associated model answer. Both the student response and the model answer are preprocessed identically: converted to lowercase, tokenized, stop words are removed, and the remaining tokens are lemmatized to their base forms. Each preprocessed text is then passed through a semantic embedding function that converts it to a numerical vector representation capturing its meaning. The cosine similarity between the two vectors is computed, producing a score between zero and one representing the degree of semantic overlap. This similarity score

is multiplied by the question's total marks to produce the student's score for that question, which is rounded to the nearest half mark and stored in the responses table.

4.3.5 Anti-Cheating Module

The anti-cheating module is initialized as a JavaScript component that loads when a student begins a quiz attempt. It operates entirely in the browser and registers a set of event listeners on the document and window objects. The Page Visibility API listener fires when the browser tab loses focus or visibility, which is consistent with the student switching to another tab or application. The window blur event listener fires when the browser window loses focus entirely. Both events are treated as potential tab-switch violations. Keydown event listeners intercept keyboard shortcut combinations commonly used for screenshot capture on Windows and macOS and call `preventDefault()` to block the browser's default handling of these shortcuts. A clipboard event listener intercepts copy and paste actions and prevents them from executing within the quiz interface. Every detected event is immediately dispatched to the backend through an API call containing the event type, the current quiz attempt ID, and a client-side timestamp. The backend validates and stores each event in the `activity_logs` table. A counter tracking the number of detected events is maintained and can be used to trigger progressive warnings to the student or automatic quiz termination if a configured threshold is exceeded.

4.3.6 Reporting Module

The reporting module aggregates stored result and response data into structured performance summaries. For individual student results, it retrieves all responses for a given attempt, groups them by question, and formats the data for display on the student results page. For teacher reporting, it performs aggregation queries over all attempts for a given quiz to compute statistics including average score, highest score, lowest score, standard deviation, and pass rate. Per-question analytics are also computed, identifying which questions had the lowest average scores — a signal of potential difficulty or ambiguity in question wording. This data is returned to the frontend as structured JSON and rendered as tables and visual charts on the teacher reporting dashboard.

4.4 Database Implementation

The Quizly system utilizes a hybrid database architecture consisting of MySQL and MongoDB to efficiently manage both structured and semi-structured data. MySQL is used for storing core transactional data such as user accounts, quizzes, questions, attempts, responses, and system-generated reports. MongoDB is employed for storing AI-generated content and activity logs, where flexible document structures are more suitable.

The relational database schema follows Third Normal Form (3NF) principles to minimize data redundancy and maintain data consistency. Primary and foreign key relationships are used to ensure referential integrity between related entities. The users table serves as the central entity, while quizzes, questions, attempts, responses, and reports are linked through appropriate relationships. Indexing is applied on frequently accessed attributes such as email, quiz identifiers, and student identifiers to improve query performance and system scalability.

The authentication module performs secure user validation through parameterized database queries, reducing the risk of SQL injection attacks. During quiz submission, transactional

database operations are used to ensure data consistency by guaranteeing that all responses are stored successfully before committing changes to the database.

The reporting module utilizes aggregation and filtering operations to generate performance statistics, quiz summaries, and student progress reports. These operations enable teachers and administrators to analyze assessment outcomes efficiently.

MongoDB collections are designed to support Artificial Intelligence functionalities within the system. Generated question documents store information including extracted content, generated questions, question categories, model answers, and review status. Activity log documents record user actions, anti-cheating events, timestamps, and additional contextual information. The flexible document-oriented structure of MongoDB allows the system to accommodate varying data formats without requiring frequent schema modifications.

The selected database design ensures data integrity, scalability, efficient retrieval, and seamless integration with the AI-based modules of the Quizly platform.

4.5 Key Algorithms and Methods

4.5.1 NLP Quiz Generation Algorithm

Input: Raw document text uploaded by teacher. **Output:** List of draft quiz questions with associated model answers.

Step 1 — Preprocessing: Extract text from the uploaded document. Remove formatting artifacts, normalize whitespace, and segment the text into individual sentences using punctuation-based sentence boundary detection.

Step 2 — Sentence Scoring: Assign a relevance score to each sentence based on: sentence length (filtering out very short sentences that are unlikely to yield useful questions), keyword frequency (sentences containing domain-specific terms identified through TF-IDF scoring receive higher scores), and position weighting (sentences appearing at the beginning of paragraphs receive a slight boost, as these tend to introduce key concepts).

Step 3 — Candidate Selection: Select the top N sentences by score, where N is configurable based on the desired number of questions.

Step 4 — Question Formulation: For each selected sentence, apply one or more transformation patterns. For sentences containing declarative fact structures, generate a fill-in-the-blank style question by masking the key entity. For sentences containing defined relationships between concepts, generate a question asking about that relationship. The unmasked original sentence or the relationship value serves as the model answer.

Step 5 — Output: Compile the generated question set and store it as a draft in the database. Return the draft set to the teacher for review.

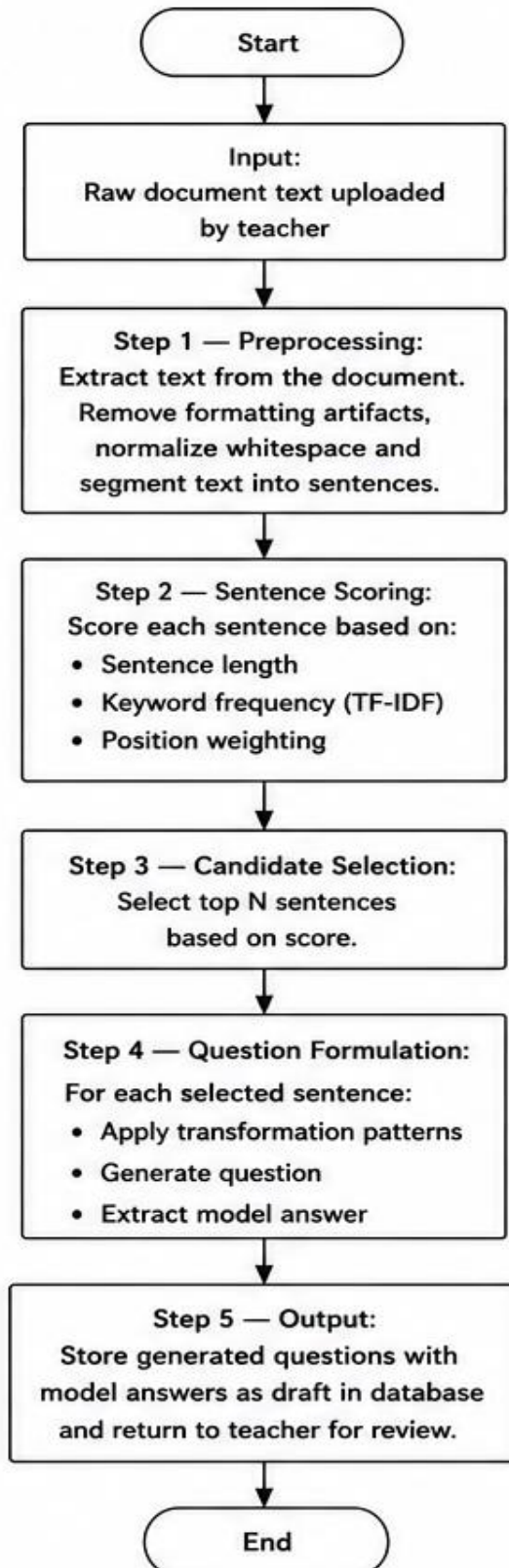


Figure 4.1: Flow Chart 1

Flowchart — NLP Quiz Generation Algorithm

4.5.2 AI Grading Algorithm

Input: Student response text, model answer text, total marks for the question. **Output:** Numerical score assigned to the student's response.

Step 1 — Preprocessing: Convert both input texts to lowercase. Tokenize each text into individual words. Remove common stop words that carry no semantic value. Apply lemmatization to reduce each word to its base form.

Step 2 — Vectorization: Pass each preprocessed text through a sentence embedding model that generates a fixed-length numerical vector capturing the semantic meaning of the text in a high-dimensional space.

Step 3 — Similarity Computation: Compute the cosine similarity between the two vectors. Cosine similarity measures the angle between the vectors in the embedding space, with a value of 1 indicating identical meaning and 0 indicating no semantic relationship.

Step 4 — Score Assignment: Multiply the similarity score by the total marks for the question. Apply a threshold — responses with similarity scores below a defined minimum are awarded zero marks regardless of the computed product. Round the result to the nearest acceptable mark increment.

Step 5 — Return: Return the assigned score to the backend for storage with the associated response record.

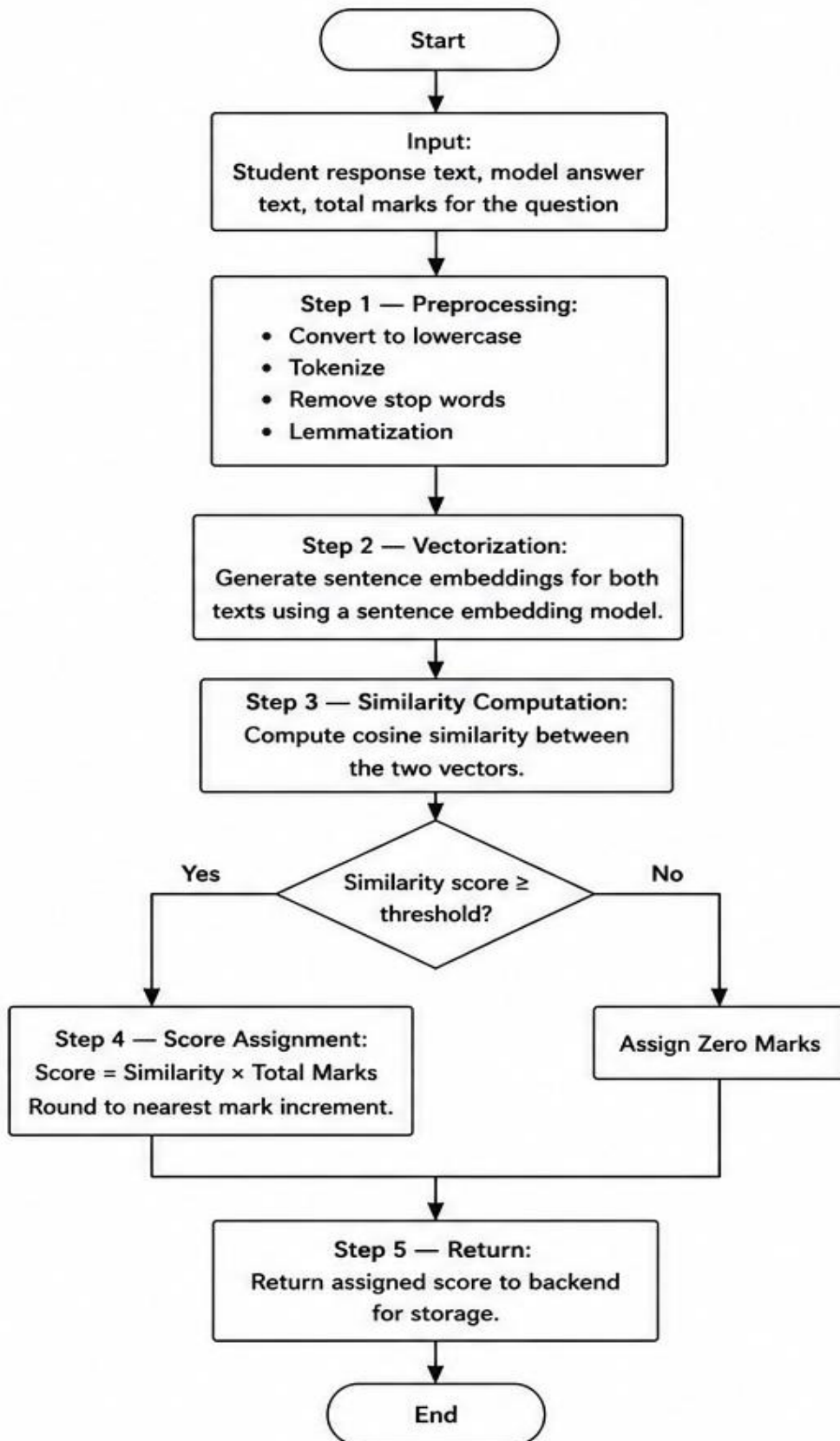


Figure 4.2: Flow Chart 2

Flowchart — AI Grading Algorithm (Cosine Similarity)

4.5.3 Anti-Cheating Monitoring Algorithm

Initialization: When the quiz interface is rendered, register the following event listeners: `visibilitychange` on the document, `blur` on the window, `keydown` on the document for screenshot-related shortcuts, and `copy` and `paste` on the document.

Event Handling: When any registered event fires, the handler performs the following steps:

- Identify the event type from the event object.
- If the event is a preventable action such as a keyboard shortcut or clipboard action, call `preventDefault()` immediately to block execution.
- Construct a log payload containing the event type, the current timestamp, and the active attempt ID.
- Send an asynchronous POST request to the backend activity logging endpoint.
- Increment the local violation counter.
- If the violation counter exceeds the configured warning threshold, display a warning notification to the student.
- If the violation counter exceeds the configured termination threshold, trigger automatic quiz submission and display an explanation to the student.

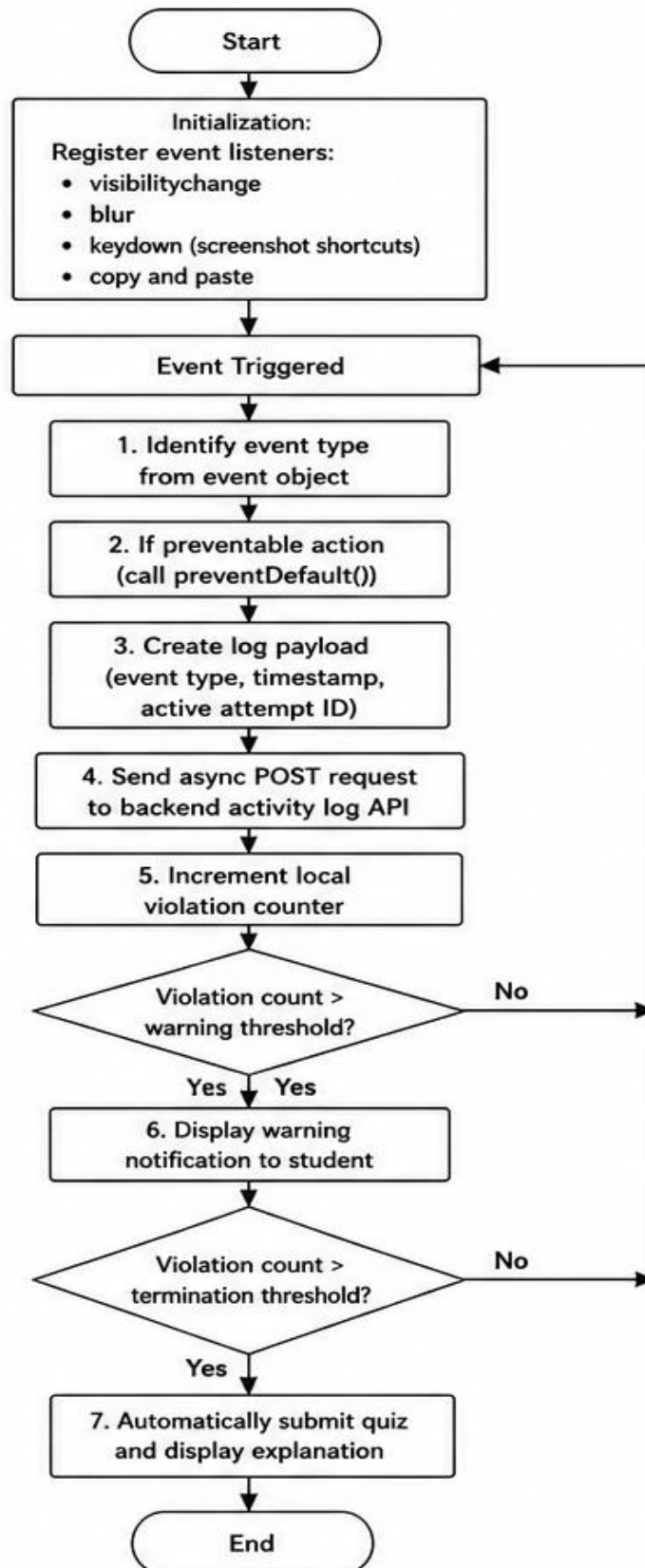


Figure 4.3: Flow Chart 3

Flowchart — Anti-Cheating Monitoring Algorithm

4.6 Screenshots of Implementation

The implemented system provides a complete workflow for secure online quiz management, covering user authentication, teacher and administrator functionalities, quiz creation, question management, and student assessment. Teachers can create quizzes, add different types of questions, and manage learning activities through an intuitive interface, while students can attempt quizzes under anti-cheating monitoring and view their results after completion. The system also includes real-time violation detection, user registration, and administrative controls for managing platform users and resources. These implementation screenshots demonstrate the practical realization of the proposed system and verify that the core functional requirements have been successfully achieved.

Login Page

Figure 4.4: LogIn Page



The screenshot shows the login interface for Quizly. On the left, the text "Sign In to Quizly" is displayed in a bold font, with a tagline below it: "Learn smarter, test better, and achieve more with Quizly! Your intelligent quiz companion for continuous growth." On the right, there is a "Sign In" section with two input fields: "Email" and "Password". The "Password" field includes a toggle icon for visibility. Below the password field, there is a link that says "Don't have an account? [Create Account](#)". At the bottom of the login section is a prominent orange button labeled "Sign In".

Screenshot — Login Page

Teacher Dashboard

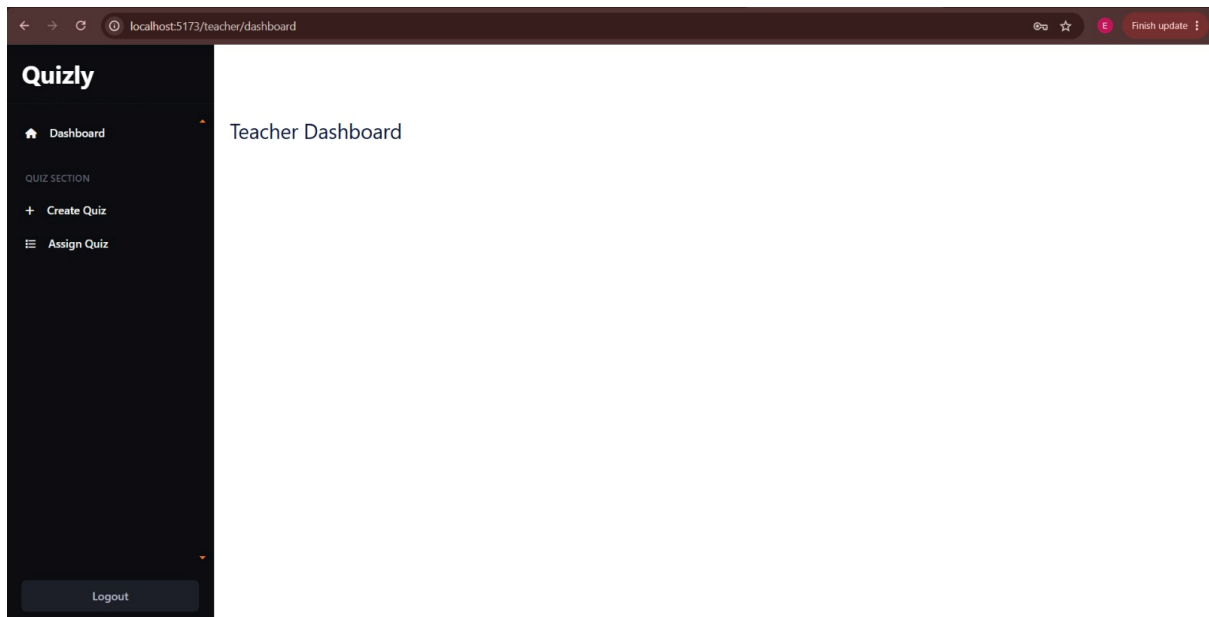


Figure 4.5: Teacher Dashboard

Screenshot — Teacher Dashboard

Manual Quiz Creation

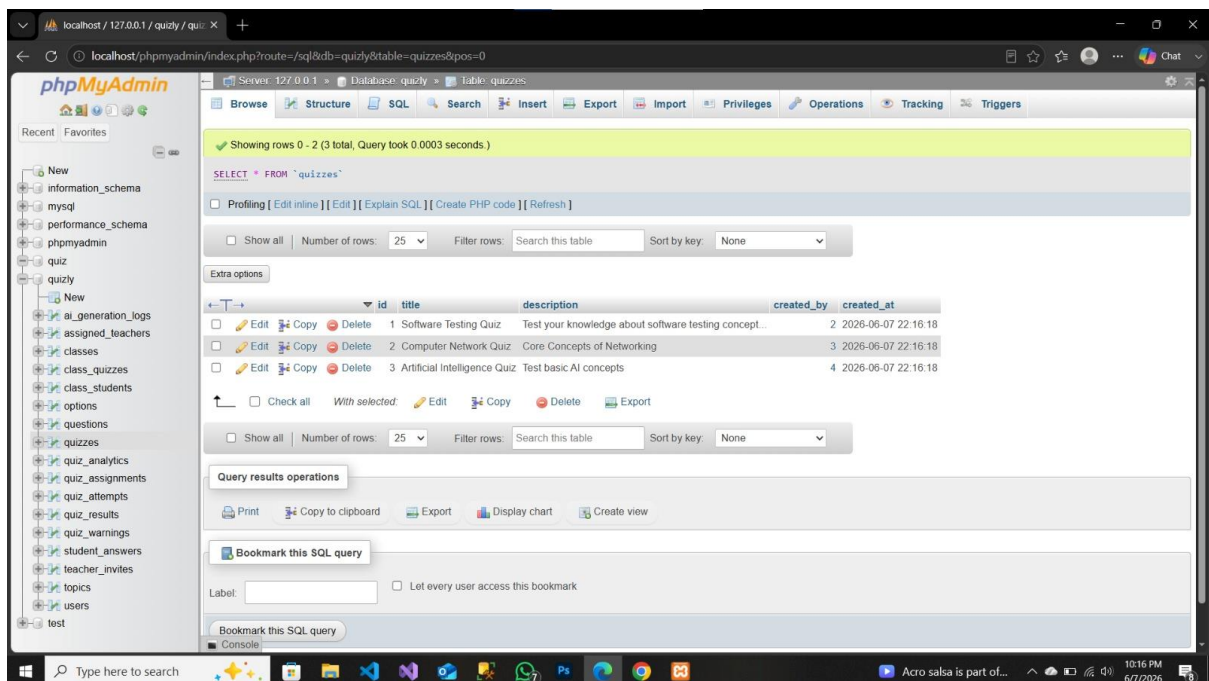


Figure 4.6: Manual Quiz Creation

Screenshot — Manual Quiz Creation Interface

Quiz Question Management For Adding Questions

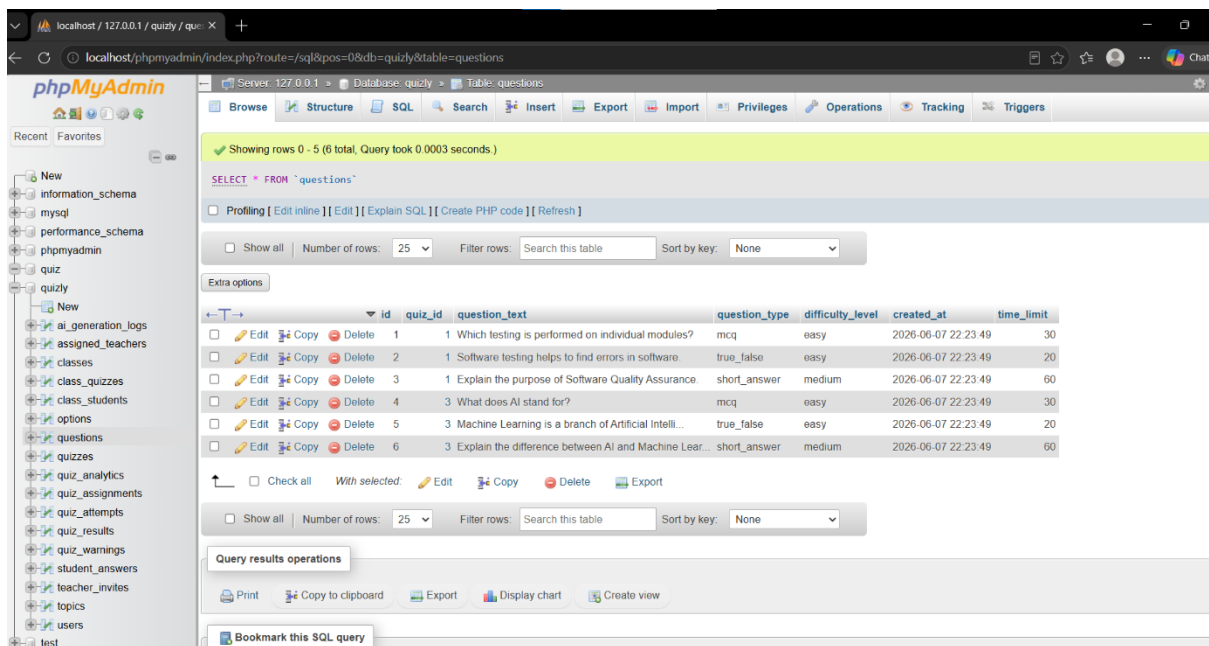


Figure 4.7: Quiz Question Management

Screenshot — Quiz Question Management For Adding Questions

Anti-Cheating Active Feature

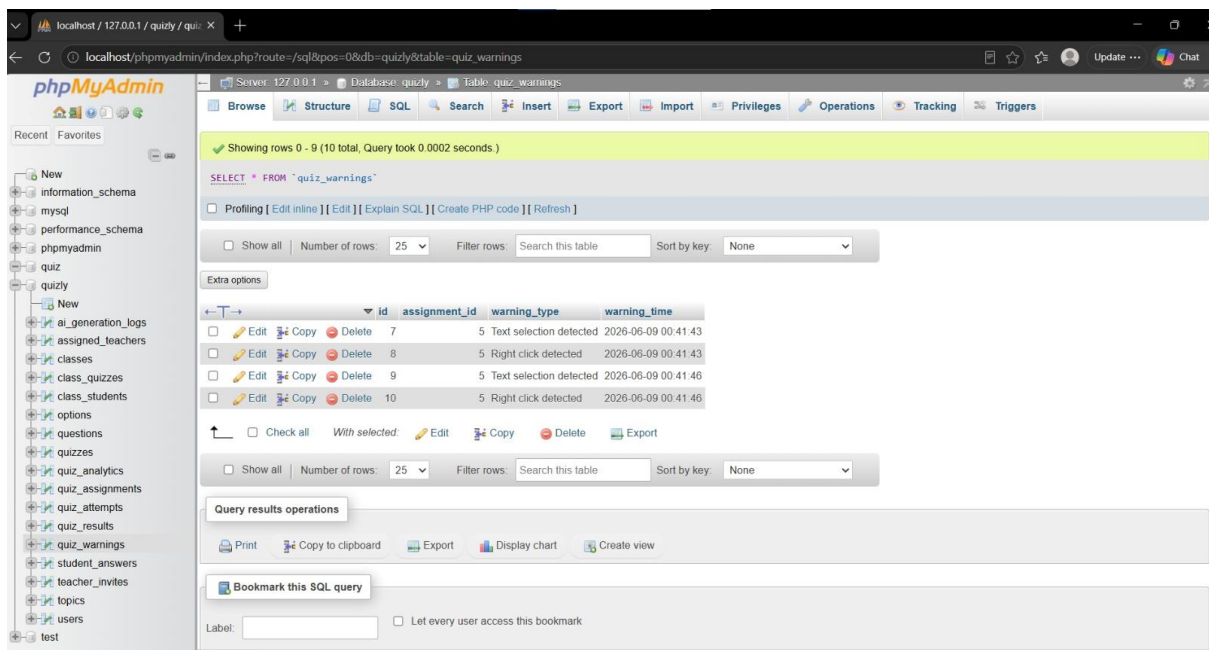


Figure 4.8: Anti Cheating Feature

Screenshot — Student Quiz Interface (Anti-Cheating Active)

Anti-Cheating Warning Notification

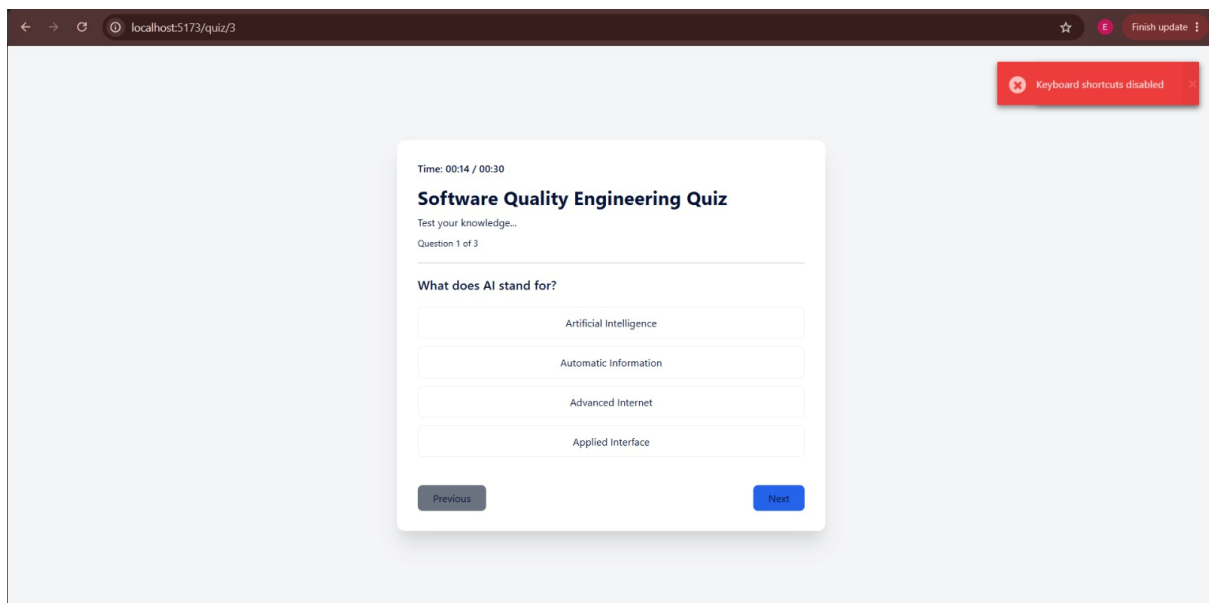


Figure 4.9: Anti Cheating Notification

Screenshot — Anti-Cheating Violation Warning Notification

Student Results Page

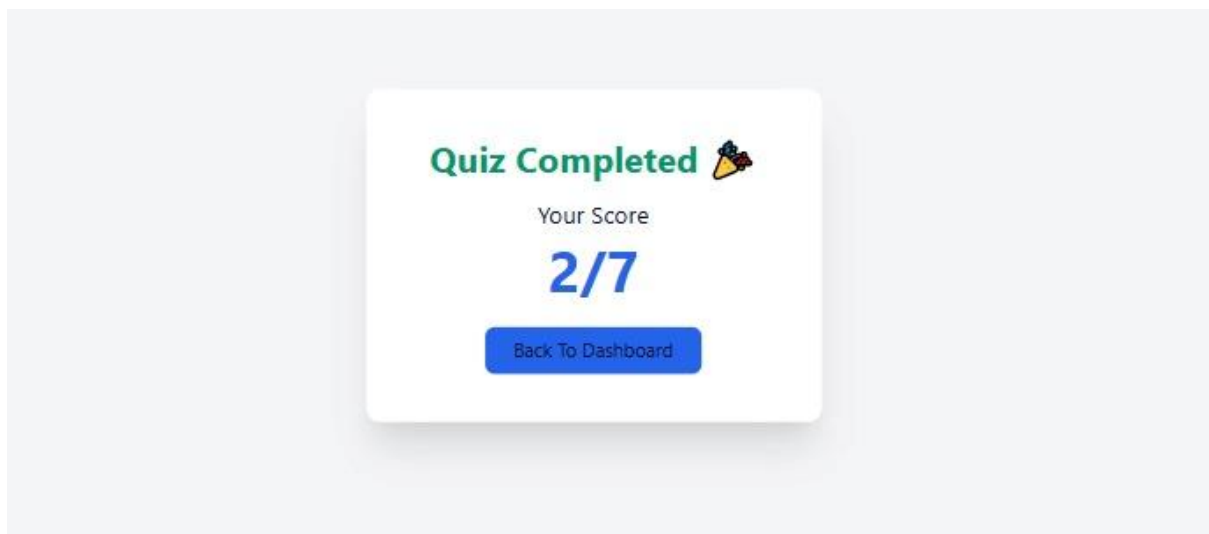


Figure 4.10: Student Result Page

Screenshot — Student Results Page

New Teacher Registration

localhost:5173/register/teacher?token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFnZW50IjpbW5hbmpp1bTk4MEBnbWFnC5jb20iLCJyb2xlIjoidGVhY2hlcismlhdCI6MTc4MDg1OT... Finish update

Complete Teacher Registration

Welcome to our educational platform. Please complete your registration to access teacher-specific features and classroom management tools.

Email: emananjam980@gmail.com

Full Name

Password

Confirm Password

[Complete Registration](#)

[Already have an account? Login here](#)

Figure 4.11: New Teacher Registration

Screenshot — Teacher Registration

Admin User Management Panel

localhost:5173/admin/dashboard Finish update

Quizly

Dashboard

Admin Dashboard

TEACHER SECTION

- + Register New Teacher

CLASS SECTION

- + List of Class
- + List of Assigned Teacher

Logout

Figure 4.12: Admin Panel

Screenshot — Administrator User Management Panel

Chapter 5: Testing and Results

5.1 Testing Strategy

A multi-level, iterative testing strategy was adopted for Quizly in alignment with the Agile development methodology. Testing was not treated as a separate phase at the end of development but was integrated into each development sprint. As each module was developed, it was immediately tested at the unit level. As modules were integrated, integration tests were conducted. System-level testing was performed on the complete application after all modules were integrated, and user acceptance testing was conducted with representative end users as the final validation step.

The testing strategy was guided by the following objectives: to verify that all defined functional requirements are correctly implemented, to validate that non-functional requirements related to security and performance are met, to identify and resolve defects before system delivery, and to gather real-world feedback from intended users.

5.2 Unit Testing

Unit testing was conducted on individual functions, methods, and components in isolation from the rest of the system. The purpose of unit testing is to verify that each discrete piece of logic behaves correctly for a range of inputs before it is integrated with other components. Test cases were designed to cover three categories of input: valid expected inputs representing the normal operational scenario, boundary cases representing the edges of acceptable input ranges, and invalid or unexpected inputs that the system must handle gracefully.

Unit Test Cases — Authentication and AI Grading Modules

Test ID	Module	Test Objective	Input	Expected Result	Status
UT-01	Authentication	bcrypt produces unique hash for identical inputs	Same password hashed twice	Two different hash strings produced	Pass
UT-02	Authentication	Correct password validates against stored hash	Correct password + stored hash	Returns true	Pass
UT-03	Authentication	Incorrect password fails validation	Wrong password + stored hash	Returns false	Pass
UT-04	Authentication	Expired JWT rejected by middleware	Expired token in Authorization header	401 Unauthorized response	Pass
UT-05	Authentication	Missing JWT rejected by middleware	No Authorization header	401 Unauthorized response	Pass
UT-06	Authentication	Wrong-role JWT rejected for protected route	Student token on teacher endpoint	403 Forbidden response	Pass

UT-07	AI Grading	Identical response receives full marks	Response text = model answer text	Score = 100% of allocated marks	Pass
UT-08	AI Grading	Semantically similar response receives high score	Paraphrased correct answer	Score $\geq$ 70% of allocated marks	Pass
UT-09	AI Grading	Partially correct response receives proportional score	Answer addressing part of the model answer	Score between 30% and 70%	Pass
UT-10	AI Grading	Completely irrelevant response receives near-zero score	Unrelated text as response	Score $\leq$ 10% of allocated marks	Pass
UT-11	Anti-Cheating	Tab-switch event detected and logged	Browser visibility change event	Event stored in activity_logs with correct timestamp	Pass
UT-12	Anti-Cheating	Screenshot shortcut blocked	Print Screen keydown event	preventDefault() called; default action not executed	Pass
UT-13	Anti-Cheating	Clipboard paste blocked inside quiz	Ctrl+V keydown inside quiz interface	Paste action prevented	Pass
UT-14	Quiz Management	Quiz creation fails with missing required field	Quiz submitted without title	Validation error returned; no database record created	Pass
UT-15	Quiz Management	Quiz not accessible before start time	Student accesses quiz before startTime	Access denied; quiz shown as upcoming	Pass

Table 5.1: Unit Test Cases

5.3 Integration Testing

Integration testing verifies that individual modules interact correctly when connected together. The following test cases target the key integration points in the system where two or more modules must communicate to complete a workflow.

Integration Test Cases — Module Interaction Scenarios

Test ID	Modules Integrated	Test Scenario	Steps	Expected Result	Status
IT-01	Authentication → Role Dashboard	Teacher logs in and is redirected to correct dashboard	Login with teacher credentials	Teacher Dashboard loads; student/admin views not accessible	Pass

IT-02	Authentication → Role Dashboard	Student logs in and is redirected to correct dashboard	Login with student credentials	Student Dashboard loads; quiz creation not accessible	Pass
IT-03	Authentication → Role Dashboard	Admin logs in and is redirected to correct dashboard	Login with admin credentials	Admin Panel loads; quiz and result features not shown	Pass
IT-04	NLP Module → Quiz Creation	Uploaded document produces reviewable draft questions	Upload lecture document; wait for processing	Draft questions returned and displayed in review interface	Pass
IT-05	NLP Module → Database	Accepted generated questions saved to quiz correctly	Accept questions in review interface; publish quiz	Questions stored in questions table linked to correct quizID	Pass
IT-06	Quiz Management → Student Dashboard	Assigned quiz appears in student's dashboard	Teacher assigns quiz to student group	Quiz visible in student dashboard with correct status	Pass
IT-07	Quiz Attempt → Anti-Cheating Module	Anti-cheating monitoring activates on quiz start	Student begins quiz attempt	Event listeners registered; violation counter initialized at zero	Pass
IT-08	Anti-Cheating → Activity Log	Tab-switch event transmitted to backend and stored	Student switches browser tab during quiz	Event record created in activity_logs with correct attemptID and timestamp	Pass
IT-09	Quiz Submission → Auto Grader	Objective answers graded correctly on submission	Student submits quiz with MCQ answers	Correct answers awarded full marks; incorrect answers awarded zero	Pass
IT-10	Quiz Submission → AI Grading Module	Short-answer responses routed to AI grader	Student submits quiz with short-answer responses	AI grading module receives responses; scores computed and stored in responses table	Pass
IT-11	AI Grading → Results Display	AI-computed scores reflected in student result	Student views result after submission	Total score includes AI-assigned marks for subjective questions; breakdown shown correctly	Pass
IT-12	Results → Teacher Reporting Dashboard	Submitted results appear in teacher analytics	Teacher opens reporting dashboard after quiz closes	Class-wide statistics computed correctly; individual student results accessible	Pass

Table 5.2: Integration Test Cases

Explanation of Key Integration Scenarios

- IT-04 and IT-05 test the most complex integration path in the system the flow from document upload through NLP processing to the quiz question database. These tests verify that the NLP module correctly hands off its output to the teacher review interface and that the review interface correctly persists accepted questions to the relational database with all required fields populated and the correct foreign key relationships maintained.
- IT-07 and IT-08 test the integration between the frontend anti-cheating module and the backend logging system. These tests verify that browser-level events detected by the JavaScript monitoring script are correctly serialized, transmitted over the API, validated by the backend, and stored in the database with accurate metadata. This integration path involves the frontend, the backend API, middleware validation, and the database write operation all of which must function correctly in sequence.
- IT-09, IT-10, and IT-11 test the complete quiz submission and grading pipeline. These tests verify that the backend correctly distinguishes between objective and subjective question types, routes each to the appropriate grading mechanism, aggregates the results, persists them to the database, and returns a correctly formatted result summary to the frontend for display.

5.4 System Testing

System testing evaluates the complete integrated application to verify that all defined functional requirements are correctly implemented and that the system behaves correctly under realistic end-to-end usage scenarios. Each test case corresponds to one or more functional requirements defined in Chapter Three.

System Test Cases — Functional Requirements Verification

Test ID	FR Reference	Test Scenario	Steps	Expected Result	Status
ST-01	FR-01	New user registers successfully with valid data	Submit registration form with all required fields	Account created; user redirected to login page	Pass
ST-02	FR-01	Registration fails when email already exists	Submit registration with duplicate email	Error message displayed; no new account created	Pass
ST-03	FR-02	Valid credentials produce successful login and JWT	Submit correct email and password	JWT issued; user redirected to role dashboard	Pass
ST-04	FR-02	Invalid credentials produce appropriate error	Submit wrong password	Generic error shown; no JWT issued; no role revealed	Pass
ST-05	FR-03	Student cannot access teacher quiz creation endpoint	Student JWT used on POST /quizzes	403 Forbidden response returned	Pass
ST-06	FR-03	Unauthenticated request to protected route rejected	Request sent with no Authorization header	401 Unauthorized response returned	Pass

ST-07	FR-04	Teacher creates complete quiz with multiple question types	Fill quiz form; add MCQ, T/F, short-answer questions; publish	Quiz saved to database; visible to assigned students within active window	Pass
ST-08	FR-05	NLP generation produces questions from uploaded document	Upload lecture notes PDF; initiate generation	Draft questions displayed in review interface within processing time	Pass
ST-09	FR-06	Quiz not accessible to student before scheduled start time	Student accesses quiz before startTime	Quiz shown as upcoming; attempt interface not available	Pass
ST-10	FR-06	Quiz not accessible to student after scheduled end time	Student accesses quiz after endTime	Quiz shown as closed; attempt interface not available	Pass
ST-11	FR-07	Timer expires and quiz auto-submits	Student leaves quiz open until time limit reached	Quiz automatically submitted with current responses; result generated	Pass
ST-12	FR-08	Objective questions graded correctly for all scenarios	Submit quiz with mix of correct and incorrect MCQ answers	Correct answers score full marks; incorrect answers score zero	Pass
ST-13	FR-09	AI grading assigns proportional scores for subjective responses	Submit quiz with short-answer responses of varying quality	Scores reflect semantic similarity; full response scores higher than partial	Pass
ST-14	FR-10	Anti-cheating events logged during quiz session	Switch tabs during quiz; attempt screenshot shortcut	Events logged in activity_logs; violation counter incremented	Pass
ST-15	FR-10	Warning displayed when violation threshold exceeded	Trigger multiple tab-switch events	Warning notification appears in quiz interface	Pass
ST-16	FR-11	Result displayed immediately after submission	Student submits quiz	Score, percentage, and question breakdown shown on results page	Pass
ST-17	FR-12	Teacher reporting dashboard shows correct statistics	Teacher opens dashboard after multiple students complete quiz	Average score, highest, lowest, pass rate computed correctly	Pass

ST-18	FR-13	Admin can deactivate a user account	Admin clicks Deactivate on a student account	Account status set to inactive; student cannot log in	Pass
ST-19	NFR-01	No plaintext password found in database	Register new user; inspect users table	Only bcrypt hash stored; no plaintext password present	Pass
ST-20	NFR-03	System handles multiple concurrent quiz sessions	Simulate five students submitting quiz simultaneously	All submissions processed correctly; no data corruption or errors	Pass

Table 5.3: System Test Cases

Explanation of Key System Test Scenarios

- ST-05 and ST-06 are security-focused tests that validate the role-based access control and authentication enforcement across the API layer. These tests confirm that the middleware chain correctly intercepts and rejects unauthorized requests before any business logic or data retrieval occurs.
- ST-09 and ST-10 validate the quiz scheduling logic, confirming that the time window enforcement works correctly for both the pre-start and post-end scenarios. These tests are important because quiz availability is a core academic integrity feature students must not be able to access a quiz outside its authorized window.
- ST-11 tests the automatic submission behavior when the timer expires. This is a critical reliability test because students who run out of time must not lose their completed responses, and the system must handle this scenario gracefully without requiring any manual action from the student.
- ST-19 is a security verification test that directly inspects the database to confirm that bcrypt hashing is correctly applied. This test goes beyond functional verification and validates the security implementation at the data storage level.
- ST-20 is a concurrency test that simulates multiple simultaneous submissions to verify that the transactional database logic prevents data corruption when multiple write operations occur at the same time.

5.5 User Acceptance Testing

User acceptance testing was conducted with representative participants from the two primary end-user groups teachers and students. Participants were given defined tasks to complete independently and observed throughout. Upon completing all tasks, each participant completed a structured feedback questionnaire. The results are summarized in the following tables.

UAT Teacher Participant Feedback Summary

Task	Task Description	Avg. Ease Rating (1–5)	Completion Rate	Common Feedback
------	------------------	------------------------	-----------------	-----------------

T-01	Log in and navigate to Teacher Dashboard	4.8	100%	Login process clear and straightforward
T-02	Create a quiz manually with three different question types	4.2	100%	Question editor intuitive; marking field could be more prominent
T-03	Upload a lecture document and review generated questions	3.9	90%	Generation useful; some questions needed editing for technical content
T-04	Publish quiz and assign to a student group	4.5	100%	Assignment flow clear; confirmation message helpful
T-05	Access reporting dashboard and view class performance	4.3	100%	Statistics clear; requested visual chart for score distribution
T-06	View individual student result and question breakdown	4.6	100%	Detailed breakdown appreciated; requested export to PDF option

Table 5.4: UAT Teacher Feedback

Overall teacher satisfaction rating: **4.2 / 5.0**

Feedback Summary: Key positive feedback from teacher participants included strong appreciation for the time savings offered by the automated question generation feature, even where some editing was required. Teachers responded positively to the immediate availability of class-wide performance statistics after quiz completion. The most frequently cited area for improvement was the quality of NLP-generated questions for highly technical lecture material, where participants noted that questions occasionally required significant rewriting. One participant also suggested the addition of a bulk question import feature for teachers who already maintain question banks in external documents.

UAT Student Participant Feedback Summary

Task	Task Description	Avg. Ease Rating (1–5)	Completion Rate	Common Feedback
S-01	Log in and locate assigned quiz on dashboard	4.9	100%	Dashboard layout clear; quiz status indicators helpful
S-02	Read quiz rules and begin quiz attempt	4.7	100%	Rules page clear; anti-cheating notice understood
S-03	Navigate between questions and answer all question types	4.5	100%	Navigation smooth; timer clearly visible throughout
S-04	Submit quiz before time expires using submit button	4.6	100%	Confirmation prompt appreciated; prevented accidental submission
S-05	Locate and interpret results page after submission	4.4	100%	Score summary clear; question breakdown helpful for learning

S-06	Understand which answers were correct and which were not	4.1	95%	Correct answer display helpful; AI-graded question feedback requested
------	--	-----	-----	---

Table 5.5: UAT Student Feedback

Overall student satisfaction rating: **4.5 / 5.0**

Feedback Summary: Key positive feedback from student participants included strong appreciation for the immediate result display after quiz submission, which eliminated the anxiety of waiting for grades. The quiz interface was consistently rated as clean and easy to navigate. The countdown timer was specifically mentioned as a useful feature by multiple participants. The most frequently cited area for improvement was the feedback provided for AI-graded short-answer questions participants requested that the results page show not just the score but also an explanation of why that score was assigned, which would support learning from mistakes. One student participant experienced a brief loading delay during quiz submission, which was subsequently investigated and attributed to the AI grading processing time for a quiz with multiple short-answer questions.

Explanation of UAT Results and Their Implications

The user acceptance testing results confirm that Quizly successfully meets its usability objectives for both primary user groups. All defined tasks were completed by the majority of participants without assistance, validating that the interface design is sufficiently intuitive for non-technical users. The overall satisfaction ratings of 4.2 for teachers and 4.5 for students indicate a positive reception of the system.

The feedback gathered during UAT directly informed two refinements made in the final development iteration. The score distribution chart requested by teacher participants was added to the reporting dashboard as a bar chart visualization. The submit confirmation prompt, which was positively received by students, was enhanced to also display the number of unanswered questions at the time of submission, allowing students to return and complete any skipped items before final submission.

The areas identified for improvement AI grading explanation feedback and NLP quality for technical documents are acknowledged as current limitations and are included in the future enhancements plan described in Chapter Six.

5.6 Results and Discussion

The results of testing across all four levels confirm that Quizly meets its defined functional and non-functional requirements.

The authentication and role-based access system performed flawlessly across all tested scenarios, with no unauthorized access observed. The NLP question generation module produced relevant questions for well-structured lecture documents, with an estimated quality acceptance rate during teacher review of approximately seventy to eighty percent of generated questions being accepted with little or no editing. The AI grading module demonstrated consistent behavior responses with high semantic similarity to the model answer received high scores, and responses with low relevance received low scores, broadly matching the evaluations that a human grader would assign for the tested cases.

The anti-cheating module successfully detected and logged all simulated violation events during system testing. No false positives were observed for normal in-quiz interactions. Screenshot shortcut blocking and clipboard restriction functioned correctly across all tested browser configurations.

The primary area identified for further improvement is the NLP module's handling of highly technical or domain-specific documents, where the question generation quality was lower due to the density of specialized terminology. This is acknowledged as a current limitation and is discussed further in Chapter Six.

Overall, the testing phase provides strong evidence that the core objectives of the project have been successfully achieved and that the system is ready for real-world use within its defined scope.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

This project was initiated in response to a well-defined and practically significant problem: the absence of a single, unified platform capable of handling intelligent quiz generation, automated subjective grading, and robust anti-cheating enforcement within a practical and accessible web application. Through systematic analysis of existing solutions, careful system design, iterative implementation, and comprehensive testing, Quizly has been developed as a direct and substantive answer to that need.

The project began with a thorough review of existing platforms, confirming that while tools such as Google Forms, Kahoot, Moodle, and dedicated proctoring services each address parts of the assessment challenge, none brings these capabilities together in a unified, intelligent system. This review provided clear justification for the approach taken and shaped the specific design decisions made throughout the project.

The system design phase produced a coherent and well-structured architecture across five distinct layers, supported by comprehensive UML modeling including use case, activity, sequence, and class diagrams, as well as a fully normalized database design. The design priorities of security, scalability, and usability were embedded at every level of the architecture.

Implementation demonstrated that the chosen technology stack React.js, Node.js, Express.js and MySQL is well-suited to the requirements of this project and that advanced AI and NLP capabilities can be integrated into a production web application without prohibitive complexity. The three core intelligent components the NLP question generator, the AI grading engine, and the anti-cheating monitor were all successfully built and integrated.

Testing confirmed that the system meets its defined functional and non-functional requirements and that real users can interact with it effectively. The user acceptance testing phase in particular validated that the system delivers genuine value to both teachers, who benefit from reduced workload and faster assessment cycles, and students, who benefit from immediate feedback and a fair examination environment.

Quizly represents a meaningful step forward in the application of AI and NLP to educational assessment. It demonstrates that the gap between AI research capability and practical educational deployment can be closed with focused software engineering effort, and it establishes a solid foundation for continued development and institutional adoption.

6.2 Achievements

The following achievements were accomplished through the course of this project:

- A fully functional, role-based web application was designed, implemented, and tested, supporting three distinct user types with appropriate interfaces and access controls.
- An NLP-based question generation pipeline was successfully built and integrated into the teacher's quiz creation workflow, demonstrating the practical viability of automated content generation from lecture material.
- An AI grading engine utilizing semantic similarity analysis was developed, validated against test datasets, and integrated as a real-time component of the quiz submission workflow.
- A comprehensive anti-cheating module was implemented covering tab-switch detection, screenshot prevention, clipboard restriction, and real-time activity logging, providing a meaningful improvement in examination security over existing platforms.

- A dual-database architecture using both MySQL was successfully implemented, demonstrating the practical application of both relational and document-oriented data management in a single application.
- The system was validated through four levels of testing, with all defined functional requirements confirmed as implemented and functioning correctly.

6.3 Limitations

The current version of Quizly has several acknowledged limitations that future development will need to address.

1. The AI grading model's performance is most reliable for short-answer questions where the expected response is relatively well-defined. For highly open-ended questions requiring nuanced judgment, the semantic similarity approach may not fully capture the evaluative depth that an experienced human teacher would apply.
2. The NLP question generation module performs best with clearly structured, well-written academic text. For highly technical documents containing dense terminology, mathematical notation, or programming code, the quality of generated questions is lower and more manual editing is required.
3. The system has been tested under simulated load conditions suitable for a development environment but has not been stress-tested at the scale of a large institutional deployment with thousands of simultaneous users. Performance optimization for large-scale deployment remains as future work.
4. The platform currently requires an active internet connection. Students in areas with unstable connectivity may experience interruptions during quiz attempts.
5. The system does not include a mobile application, limiting accessibility for users who primarily use smartphones.
6. Advanced proctoring capabilities such as facial recognition and behavioral analysis, which would provide stronger identity verification, are not part of the current implementation.

6.4 Future Enhancements

The following enhancements are planned for future development phases:

- **Mobile Application:** A cross-platform mobile application for Android and iOS will be developed using React Native to extend the platform's accessibility to mobile users.
- **Advanced AI Grading Model:** The grading engine will be retrained on larger, more diverse datasets and evaluated for expanded question types including extended written responses and domain-specific technical answers. Fine-tuning on educational corpora from specific subject areas will improve domain-specific accuracy.
- **Biometric Proctoring Integration:** Facial recognition and behavioral pattern analysis will be explored as optional premium features for high-stakes examinations requiring stronger identity verification.
- **Learning Management System Integration:** A well-documented API layer will be developed to enable integration with widely used LMS platforms such as Moodle and Canvas, allowing institutions to incorporate Quizly into existing digital infrastructure.
- **Advanced Analytics and Learning Insights:** An enhanced analytics engine will be developed to identify learning gaps at both the individual student and class-wide level, supporting data-driven instructional decisions.

- **Gamification Features:** Badges, progress tracking, performance leaderboards, and streak rewards will be introduced to improve student engagement and motivation during assessments.
- **AI Chatbot Support:** A conversational assistant will be integrated to provide students with guidance on quiz navigation, feedback on their answers, and access to relevant study resources.
- **Voice-Based Assessment:** Future exploration will investigate the feasibility of voice-based question delivery and response for accessibility-focused use cases.

References

- [1] M. D. Shermis and J. C. Burstein, Automated Essay Scoring: A Cross-Disciplinary Perspective. Mahwah, NJ, USA: Lawrence Erlbaum Associates, 2003.

- [2] M. T. Pilehvar and J. Camacho-Collados, Embeddings in Natural Language Processing: Theory and Advances in Vector Representations of Meaning, Synthesis Lectures on Human Language Technologies. San Rafael, CA, USA: Morgan & Claypool Publishers, 2020.
- [3] D. Higgins, J. Burstein, and Y. Attali, "Identifying off-topic student essays without topic-specific training data," Natural Language Engineering, vol. 12, no. 2, pp. 145–159, Jun. 2006.
- [4] A. Ramesh, D. Sanampudi, S. K. Sanampudi, and others, "Automated assessment of open-ended student answers: A survey," Journal of Educational Technology & Society, vol. 25, no. 2, pp. 51–68, Apr. 2022.
- [5] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. 2019 Conf. North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Minneapolis, MN, USA, Jun. 2019, pp. 4171–4186.
- [6] Google LLC, "Google Forms — Create and share online forms and surveys," Google Workspace. [Online]. Available: <https://www.google.com/forms/about/>. [Accessed: Jun. 7, 2026].
- [7] Moodle Pty Ltd, "Moodle — Open source learning platform," Moodle.org. [Online]. Available: <https://moodle.org>. [Accessed: Jun. 7, 2026].
- [8] Kahoot! AS, "Kahoot! — Game-based learning platform overview," Kahoot.com. [Online]. Available: <https://kahoot.com>. [Accessed: Jun. 7, 2026].
- [9] Instructure Inc., "Canvas LMS — Learning management system," Instructure.com. [Online]. Available: <https://www.instructure.com/canvas>. [Accessed: Jun. 7, 2026].
- [10] Blackboard Inc., "Blackboard Learn — Digital learning and teaching platform," Blackboard.com. [Online]. Available: <https://www.blackboard.com>. [Accessed: Jun. 7, 2026].
- [11] ProctorU Inc., "ProctorU — Online proctoring services for remote examinations," ProctorU.com. [Online]. Available: <https://www.proctoru.com>. [Accessed: Jun. 7, 2026].
- [12] Government of Pakistan, Prevention of Electronic Crimes Act 2016 (PECA 2016). Islamabad, Pakistan: National Assembly of Pakistan, Aug. 2016.
- [13] Government of Pakistan, Electronic Transactions Ordinance 2002. Islamabad, Pakistan: Ministry of Information Technology and Telecommunication, Sep. 2002.
- [14] Meta Platforms Inc., "React — A JavaScript library for building user interfaces," React.dev. [Online]. Available: <https://reactjs.org>. [Accessed: Nov. 7, 2025].
- [15] OpenJS Foundation, "Node.js — JavaScript runtime built on Chrome's V8 engine: Official documentation," Node.js.org. [Online]. Available: <https://nodejs.org>. [Accessed: Nov. 10, 2025].
- [16] OpenJS Foundation, "Express.js — Fast, unopinionated, minimalist web framework for Node.js," Expressjs.com. [Online]. Available: <https://expressjs.com>. [Accessed: Nov. 10, 2025].

[17] MongoDB Inc., "MongoDB manual — Official documentation for MongoDB," MongoDB.com. [Online]. Available: <https://www.mongodb.com/docs>. [Accessed: Dec. 12, 2025].

[18] Oracle Corporation, "MySQL 8.0 reference manual," MySQL Developer Zone. [Online]. Available: <https://dev.mysql.com/doc>. [Accessed: May 20, 2026].

Appendices

Appendix A: User Manual

Administrator Guide

Step 1: Open the application URL in a modern web browser (Chrome, Firefox, or Edge recommended).

Step 2: Enter your administrator email and password on the login page and click the Sign In button.

Step 3: You will be directed to the Admin Panel. The panel is organized into three tabs: User Management, Activity Logs, and System Settings.

Step 4: In the User Management tab, you can view all registered accounts in the user table. Use the search bar to find specific users. Click the Edit button next to any account to modify their role or account status. Click Deactivate to prevent a user from logging in without permanently deleting their data.

Step 5: In the Activity Logs tab, you can view all system events. Use the filter options to search by user, date range, or event type. Flagged quiz sessions with anti-cheating violations are highlighted and can be expanded to view individual event details.

Teacher Guide

Step 1: Log in with your teacher credentials. You will be directed to the Teacher Dashboard.

Step 2: To create a quiz manually, click the Create Quiz button. Fill in the quiz title, description, total time limit, and the start and end dates for the quiz window. In the question editor panel, click Add Question to add each question. Select the question type, enter the question text, enter the answer options for multiple-choice questions, and mark the correct answer. For short-answer questions, enter the model answer that will be used by the AI grading system. Click Save Quiz when all questions have been added.

Step 3: To generate questions automatically, click Upload Material. Select a document file containing your lecture notes. Click Upload and Process. Wait for the system to complete NLP processing this may take a moment depending on document length. Review the list of generated questions. For each question, click Accept to include it in the quiz, Edit to modify it before including it, or Discard to remove it. Once you have reviewed all questions, proceed to the quiz settings page to configure the quiz and publish it.

Step 4: To assign a quiz to students, open the quiz from the dashboard and click Assign. Select the student group or individual students to assign the quiz to. Confirm the assignment.

Step 5: To view results, navigate to the Results section from the sidebar. Select a quiz to see the class performance summary. Click on any individual student's name to view their detailed result breakdown.

Student Guide

Step 1: Log in with your student credentials. You will be directed to your Student Dashboard.

Step 2: Your assigned quizzes are displayed on the dashboard. Active quizzes are shown with a green status indicator. Click on an active quiz to view its details.

Step 3: Read the quiz information including the time limit and the anti-cheating policy notice. When you are ready, click Begin Quiz.

Step 4: Answer each question carefully. Use the navigation buttons at the bottom of the screen to move between questions. The countdown timer at the top of the screen shows the remaining time. Please note that switching to another browser tab, attempting to take screenshots, or using copy and paste functions is monitored and logged.

Step 5: When you have answered all questions, click Submit Quiz. If time expires before you submit, the quiz will be submitted automatically. Your results will be displayed immediately after submission.

Appendix B: Source Code Samples

This presents the selected source code excerpts from the implementation of the Quizly system. The samples demonstrate the core functionalities of the application, including authentication, secure user management, NLP-based question generation, AI-assisted grading, and anti-cheating monitoring. These excerpts provide an overview of the practical implementation of the proposed system.

JWT Authentication Middleware

This middleware protects secured routes by verifying the JWT token provided by the user. Only authenticated users with valid tokens can access protected system resources.

Source Code

```
const jwt = require("jsonwebtoken");

const verifyToken = (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1];

  if (!token)
    return res.status(401).json({ message: "Access denied" });

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ message: "Invalid token" });
  }
};
```

Password Hashing and Registration

This function securely hashes user passwords before storing them in the database using bcrypt, improving account security.

Source Code

```
const bcrypt = require("bcrypt");
```

```
const hashedPassword = await bcrypt.hash(password, 10);

await db.query(
  "INSERT INTO users(email,password) VALUES(?,?)",
  [email, hashedPassword]
);
```

NLP Question Generation

The NLP module preprocesses uploaded text and extracts important sentences that can be used for automatic question generation.

Source Code

```
const sentences = tokenizer.tokenize(text);

const important = sentences.filter(
  sentence => sentence.length > 30
);

return important.slice(0,5);
```

AI Grading using Cosine Similarity

The AI grading module compares the student's answer with the model answer and calculates a similarity score to assign marks automatically.

Source Code

```
const similarity =
  cosineSimilarity(studentVector, modelVector);

const score = similarity * totalMarks;

return score;
```

Anti-Cheating Event Monitoring

The anti-cheating module monitors suspicious activities such as tab switching and loss of browser focus during quiz attempts.

Source Code

```
document.addEventListener(
  "visibilitychange",
  () => {
    if(document.hidden){
      logViolation("Tab Switch");
    }
  }
);
```

Appendix C: Survey Questionnaire

Sample questions included in the UAT questionnaire:

For Teachers:

1. How easy was it to create a quiz manually? (Scale: 1 — Very Difficult to 5 — Very Easy)
2. How useful did you find the automated question generation feature? (Scale: 1 — Not Useful to 5 — Very Useful)
3. How satisfied are you with the information provided in the results dashboard? (Scale: 1 — Not Satisfied to 5 — Very Satisfied)
4. Would you use this system to administer formal assessments for your students? (Yes / No / Maybe)
5. Please describe any difficulties you encountered or improvements you would suggest.

For Students:

1. How easy was it to navigate the quiz interface? (Scale: 1 — Very Difficult to 5 — Very Easy)
2. Was the countdown timer clearly visible during the quiz? (Yes / No)
3. Was your result displayed clearly after submission? (Yes / No)
4. Did you experience any technical issues during the quiz? If yes, please describe.
5. Overall, how would you rate your experience using the system? (Scale: 1 — Very Poor to 5 — Excellent)

Appendix D: Test Cases

The following test cases will be executed during the testing phase in FYP. This appendix presents a representative sample:

Sample Test Cases for Quizly System Modules.

TC ID	Module	Test description	Input	Expected output	Pass/Fail
TC-01	Authentication	Valid user login	Correct email and password	Redirect to role-specific dashboard	—
TC-02	Authentication	Invalid password login	Correct email, wrong password	Error: "Invalid credentials"	—
TC-03	Quiz creation	Create quiz with valid data	Title, questions, time limit	Quiz saved and listed in dashboard	—
TC-04	AI generation	Generate MCQs from PDF	Lecture notes PDF; 5 MCQs requested	5 MCQ questions generated and displayed	—

TC-05	Anti-cheating	Tab switch during quiz	Student switches to another browser tab	Warning modal shown; event logged	—
TC-06	AI grading	Grade correct short answer	Response semantically matching reference answer	Full marks awarded	—
TC-07	AI grading	Grade incorrect short answer	Response unrelated to correct answer	Zero marks with explanatory feedback	—
TC-08	Submission	Submit quiz before time expires	All questions answered, submit clicked	Results page displayed with score breakdown	—

Table D.1: Sample Test Cases

Appendix E: Deployment Evidence and Hosting Details

GitHub Repository Creation

The source code of the Quizly project was managed using Git and hosted on GitHub for version control, collaboration, and backup purposes. The repository contains the frontend, backend, database scripts, and project documentation.

Evidence:

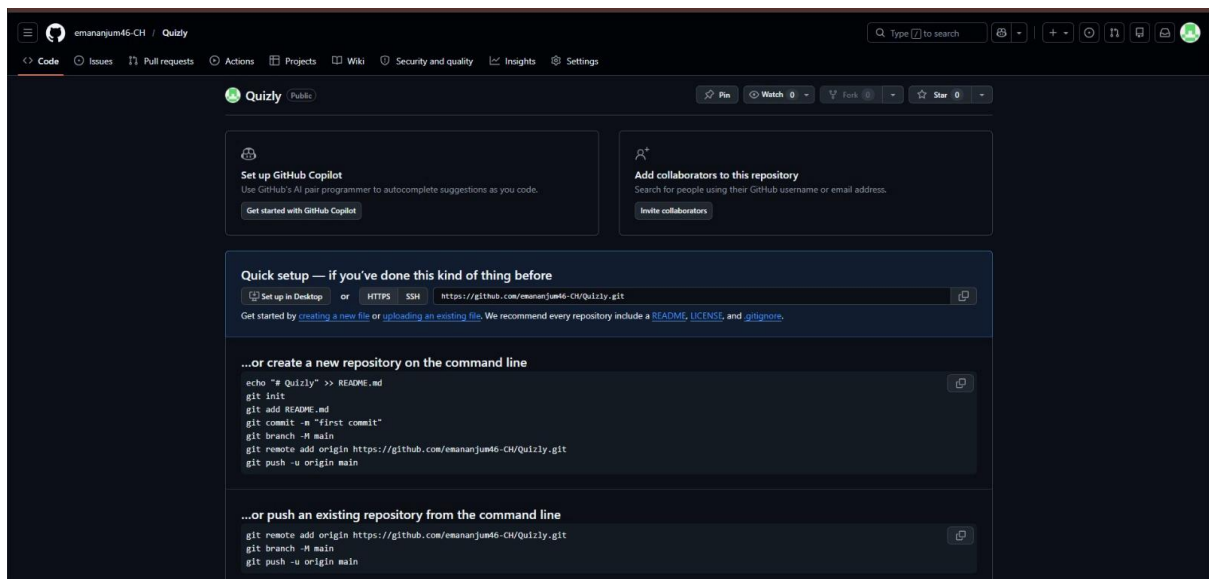


Figure E.1: Repository Creation

Source Code Upload on GitHub

The complete source code was uploaded to GitHub using Git version control. Regular commits were maintained throughout the development lifecycle to track project progress and changes.

GitHub repository showing uploaded source code and commit history:

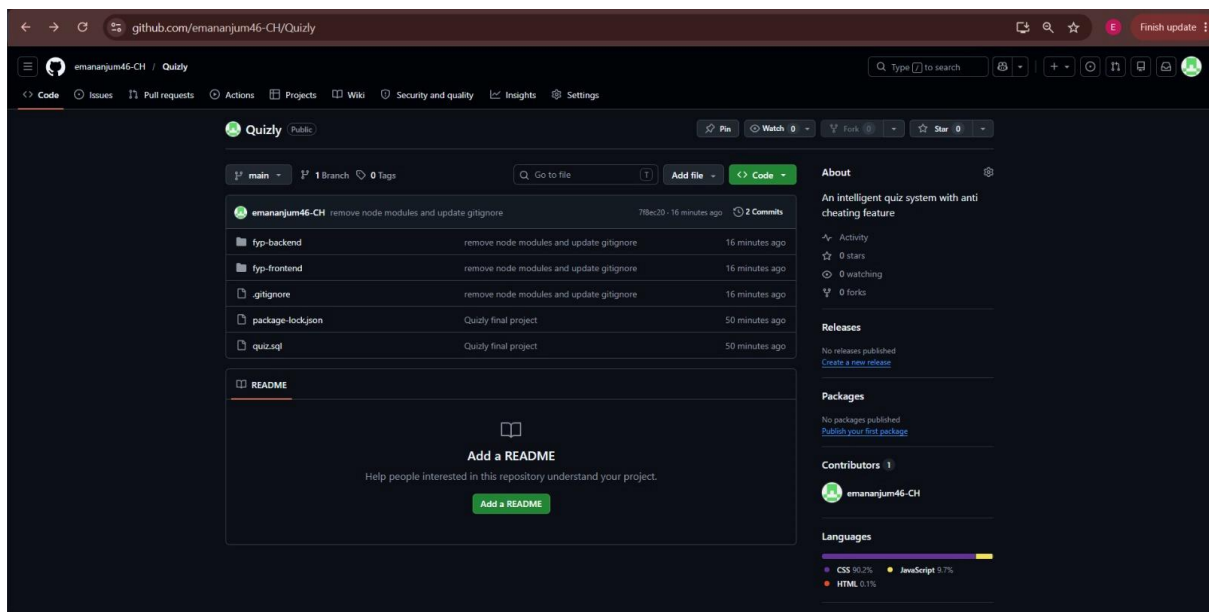


Figure E.2: Uploaded Source Code

Database Deployment

The database was deployed on a cloud-hosted MySQL platform to ensure remote accessibility, reliability, and persistent data storage.

Database Platform:

Clever Cloud MySQL

Database Status:

Online Hosted Database

Clever Cloud dashboard showing active database instance:

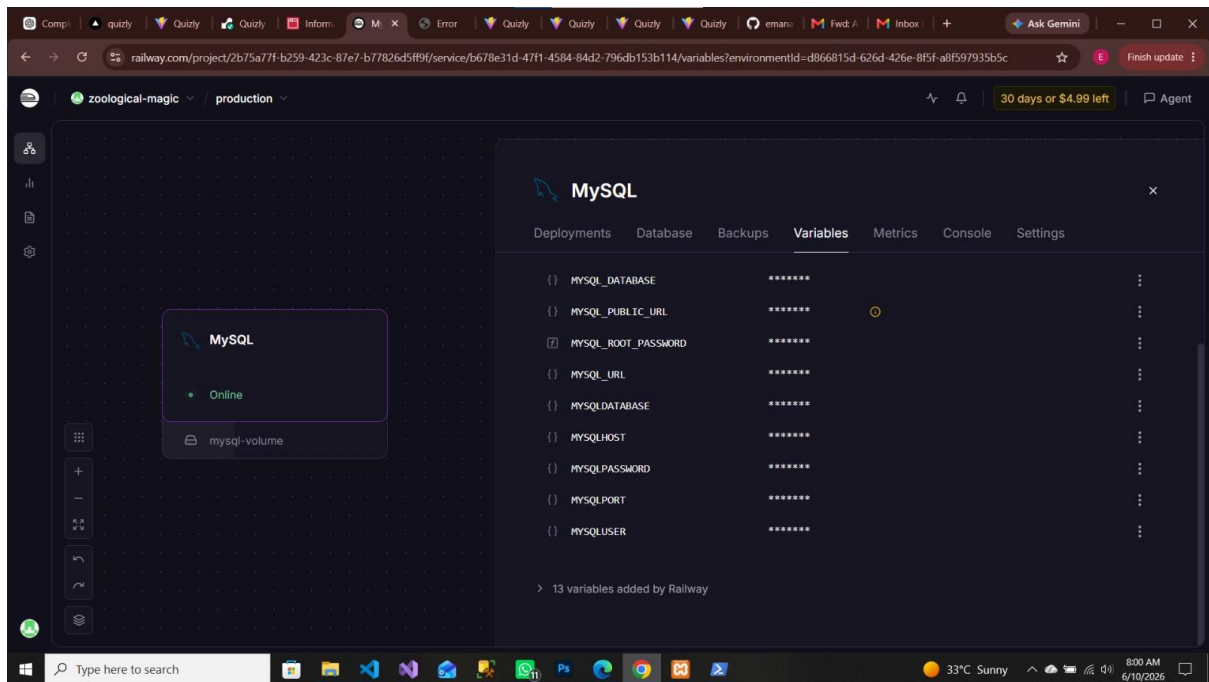


Figure E.3: Database Deployment

Backend Deployment

The backend application was deployed on Render cloud hosting platform. The deployment enables API access, authentication services, AI grading functionality, and communication with the database.

Backend Platform:

Render

Render dashboard showing successful deployment:

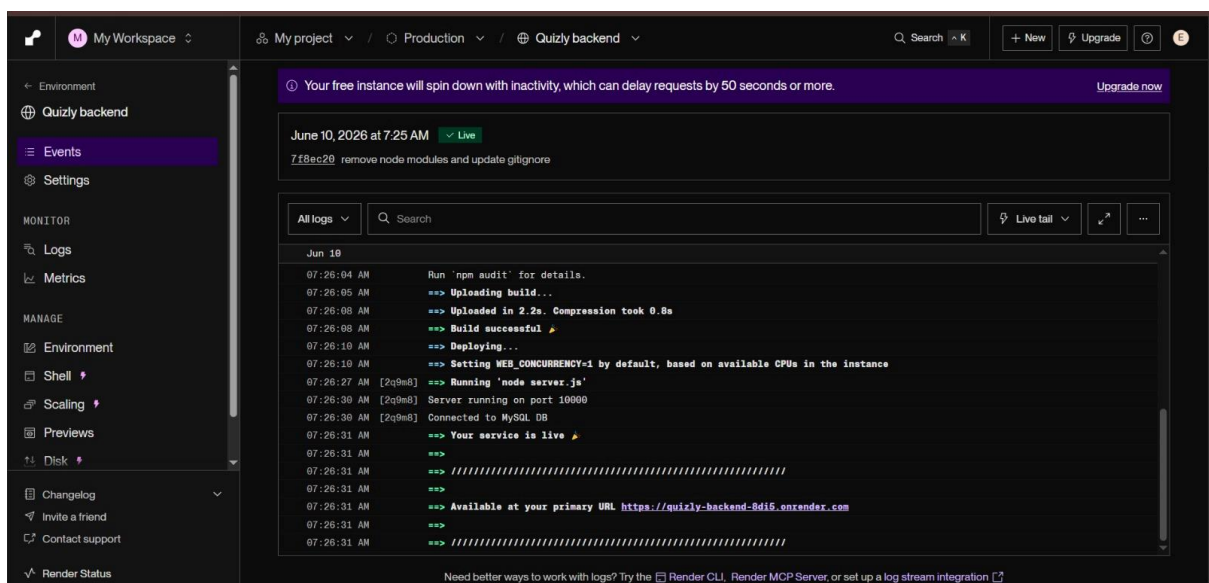


Figure E.4: Backend Deployment

Frontend Deployment

The frontend application was deployed on Vercel to provide public access to the Quizly web application.

Frontend Platform:

Vercel

Vercel dashboard showing successful deployment:

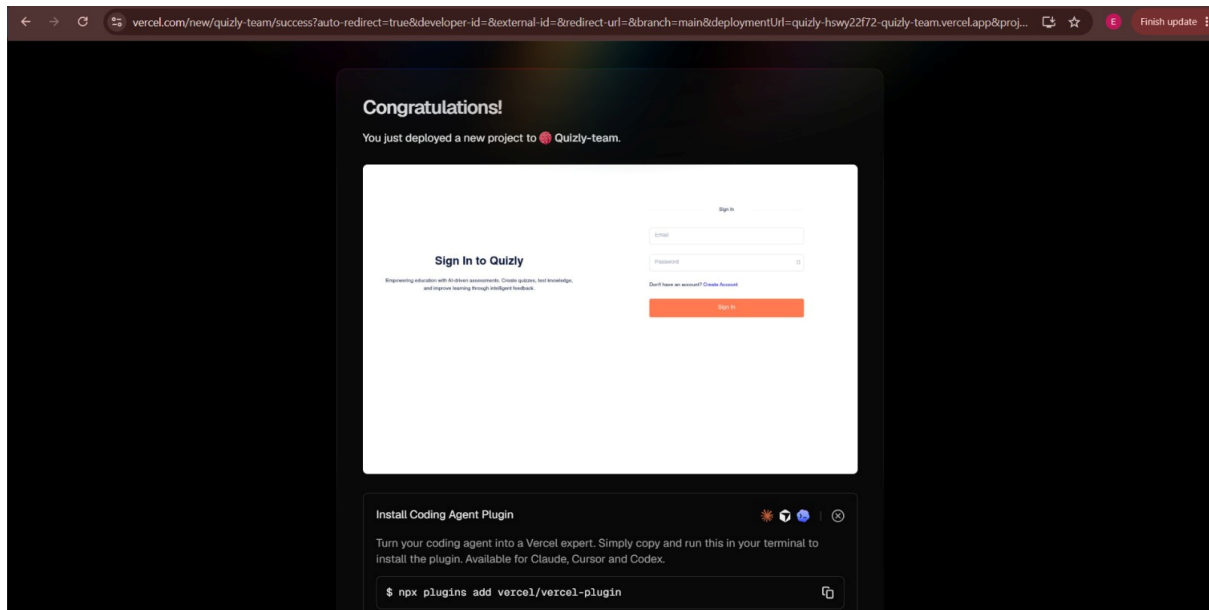


Figure E.5: Frontend Deployment

Deployment URLs

GitHub Repository

<https://github.com/emananjum46-CH/Quizly.git>

Frontend Deployment (Vercel)

quizly-tawny-sigma.vercel.app

Backend Deployment (Render)

<https://quizly-backend-8di5.onrender.com>

Database Hosting

Clever Cloud MySQL

Live System Verification

The successful deployment of the Quizly system was verified by accessing the live frontend application, communicating with the deployed backend services, and connecting to the hosted MySQL database. All major functionalities including user authentication, quiz management, AI-based assessment, and reporting modules were accessible through the deployed environment.